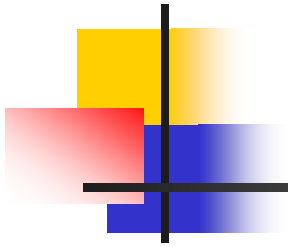
Les composants programmables : Technologie, Architecture, Synthèse

C. Belleudy (partie 2)

Université Nice SOPHIA ANTIPOLIS



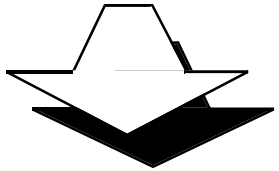
PLAN

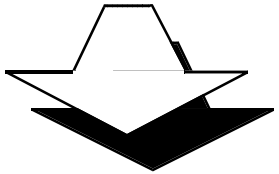
1. Introduction et Généralités sur les composants logiques programmables
2. SPLD : PAL, GAL, …
3. FPGA, CPLD : Architecture et synthèse :
    1. *Technologie et architectures*
    2. *Les CPLDs*
    3. *Les FPGAs*

Les composants logiques programmables de haute complexité

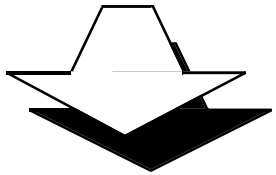
Evolution de la technologie des circuits intégrés

Demande croissante de circuits réalisant des applications de plus en plus complexes





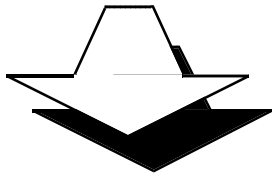
Circuit programmable de plus haute complexité :
intégration d'un plus grand nombre de portes programmables



Composants programmables de type tableau de cellules : CPLD, FPGA ....

Tableau de cellules ?

PAL : 10 cellules de sortie, 22 entrées dans la matrice ET.



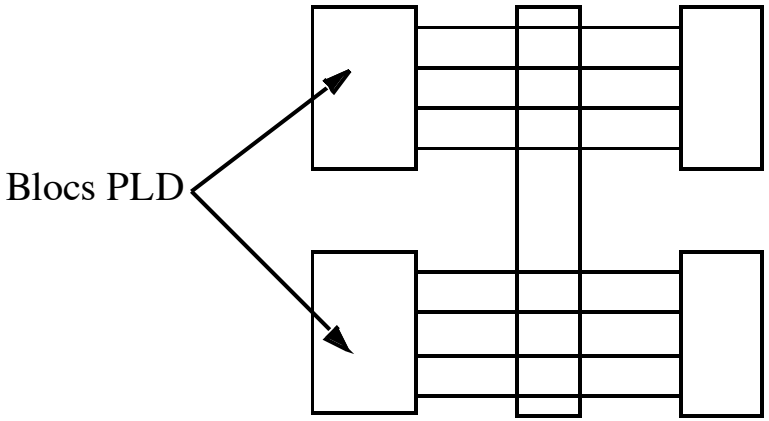
Au délà, la matrice ET devient trop importante, il est nécessaire d'adopter une nouvelle structure pour interconnecter les cellules entre elles.

Les principales sociétés qui fabriquent ce type de composants sont : Xilinx, Altera, Actel, AMD, Lattice, Texas Instruments ...

Les fabricants proposent diverses architectures aussi bien pour l'interconnexion que pour les cellules logiques.

Structure générale de ces composants

CPLD



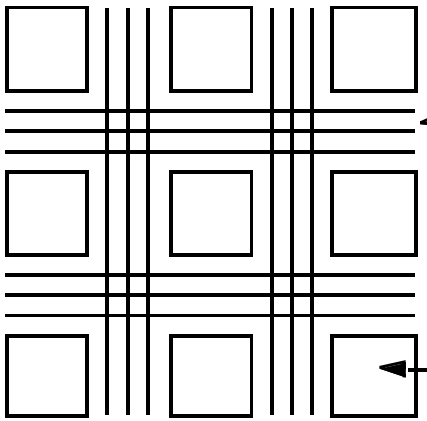
Structure hiérarchique

Performances prédictibles

FPGA

Matrice de blocs logiques et interconnexions

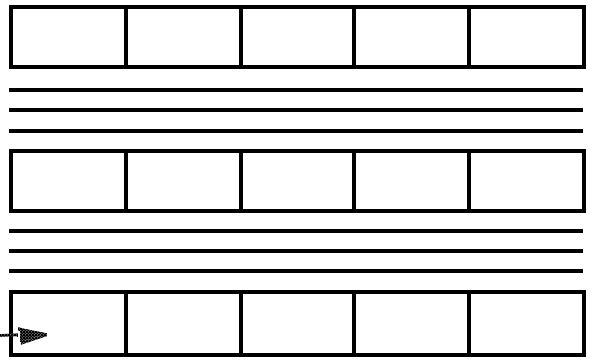
Tableau symétrique



Interconnexion

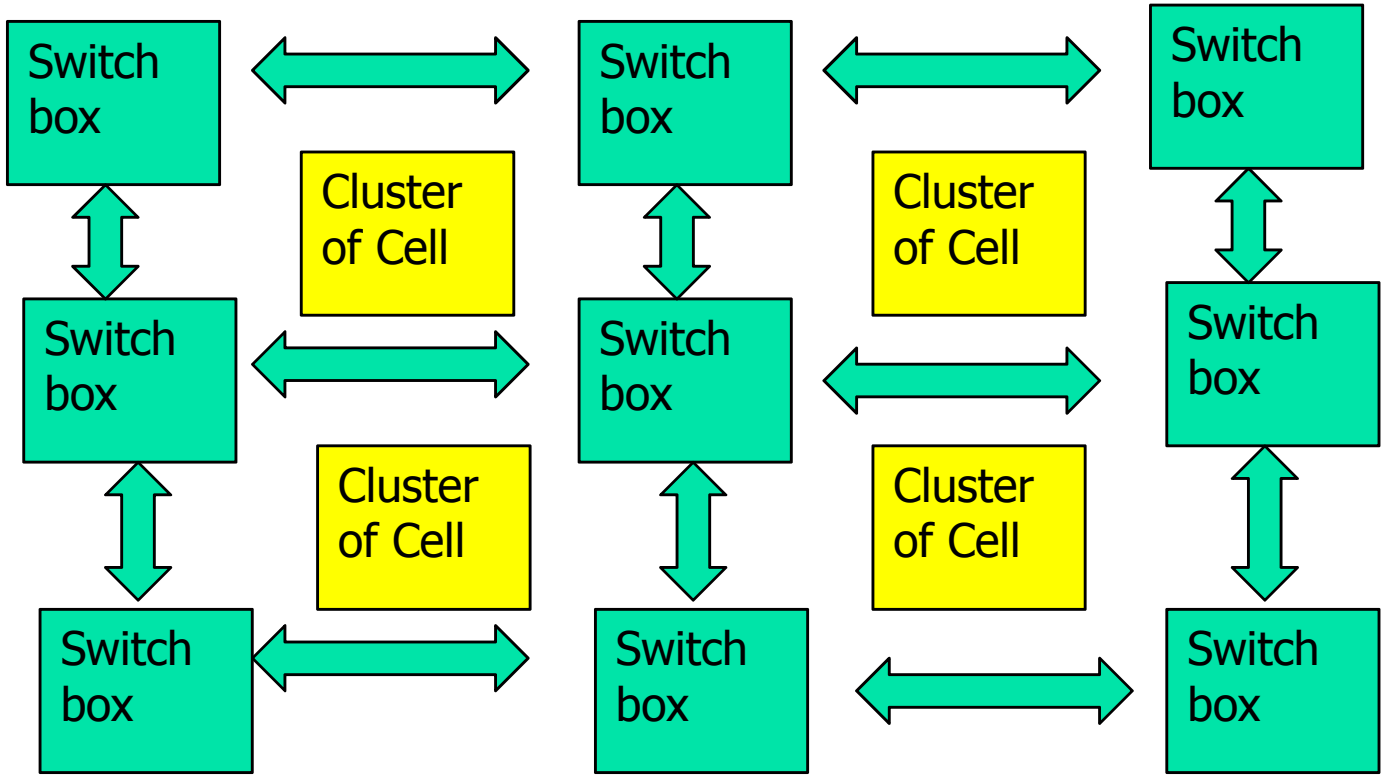
Bloc logique

Structure en lignes



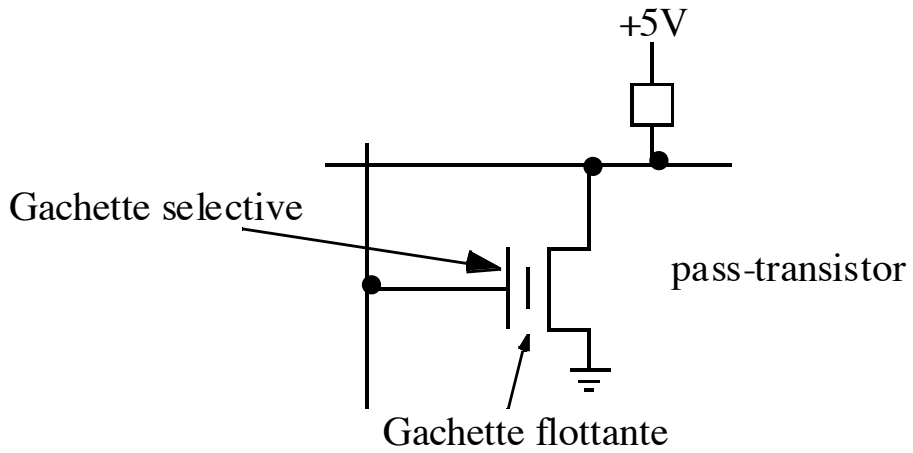
Structure générale de ces composants

FPGA hiérarchique : scalabilité



Technologie de programmation des CPLDs

Technologies EPROM et EEPROM



Nombre de programmation : de 10^3 à 10^6

Passant par défaut,

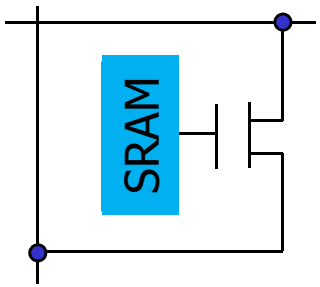
Charge sur la gâchette, le transistor est bloqué.

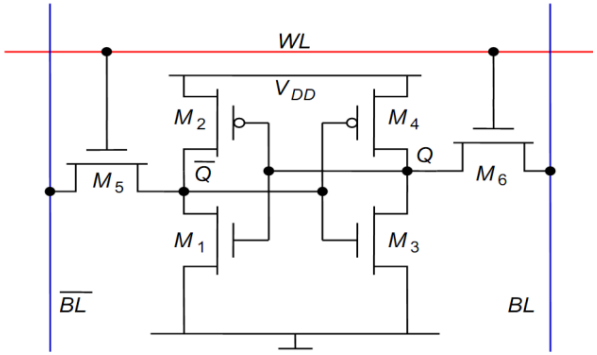
1 cellule ROM : 1 transistor (qq µm²)

Technologie de programmation des FPGAs

1 - Technologie SRAM

Programmation volatile, nécessite une ROM externe qui se charge à la mise sous tension dans la RAM interne du composant.



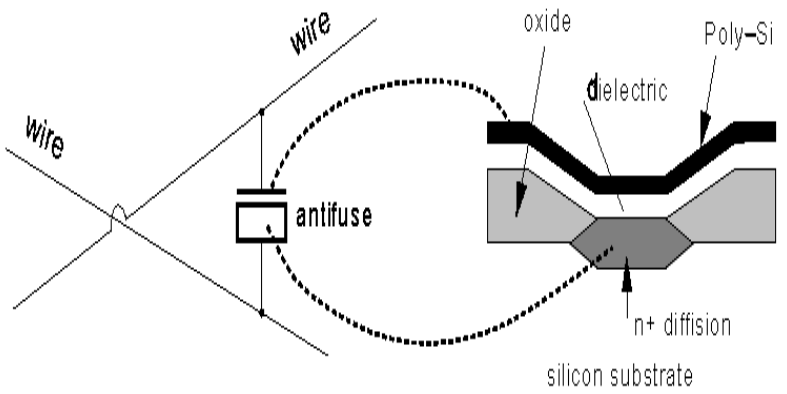


Configuration réalisée à la mise sous tension

1 cellule SRAM => 5/6 transistors
surface = qq dizaines de µm²

2 - Technologie antifusible (antifuse)

Par défaut, l'élément programmable est en haute impédance. Par application d'un voltage élevé (ex: 18V) aux bornes de l'antifusible, ce dernier devient passant (surface <1µm²)



Programmable une seule fois

Technologie de programmation

Type	EPROM	Antifusible	SRAM
Rapidité	-	+	-
Densité	-	+	--
Facilité	+	-	+
Reprogrammabilité	+	-	++

Technologie EPROM :

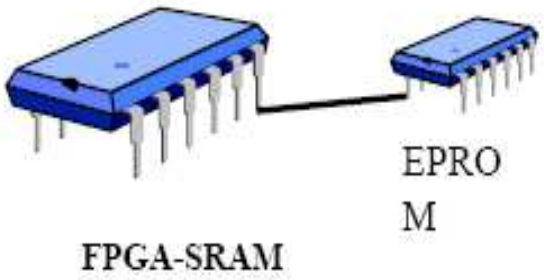
Cellules occupant dix fois plus de surface que les cellules antifusible.
Programmables plusieurs fois (10 000) par un programmateur adapté.
Possibilité de circuit ISP.

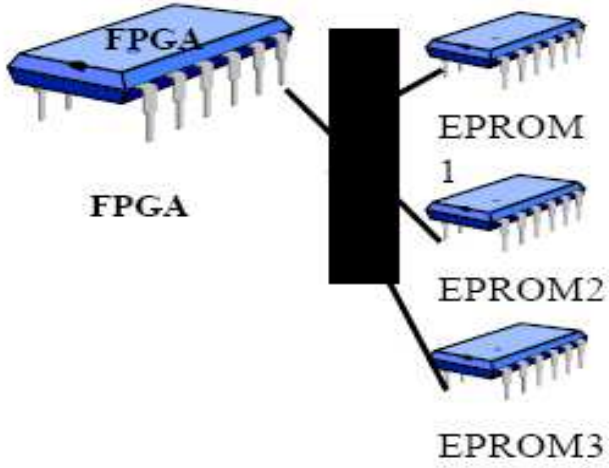
Technologie Antifusible : Programmable une seule fois (OTP).

Technologie de programmation

Technologie SRAM : Circuit ISP, nécessite une PROM de configuration.

Les cellules SRAM occupent deux fois plus de surface que les cellules EPROM







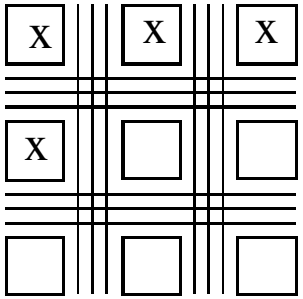
Les différentes étapes de synthèse pour les CPLDs et les FPGAs

Equations logiques

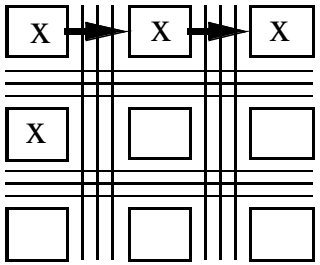
Choix du composant

Déterminer le nombre de cellules

F1 : 1 cellule
F2 : 3 cellules

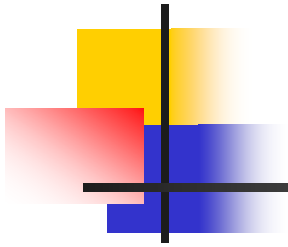


Affectation des cellules (Placement)



Routage

Programmation (bitstream)

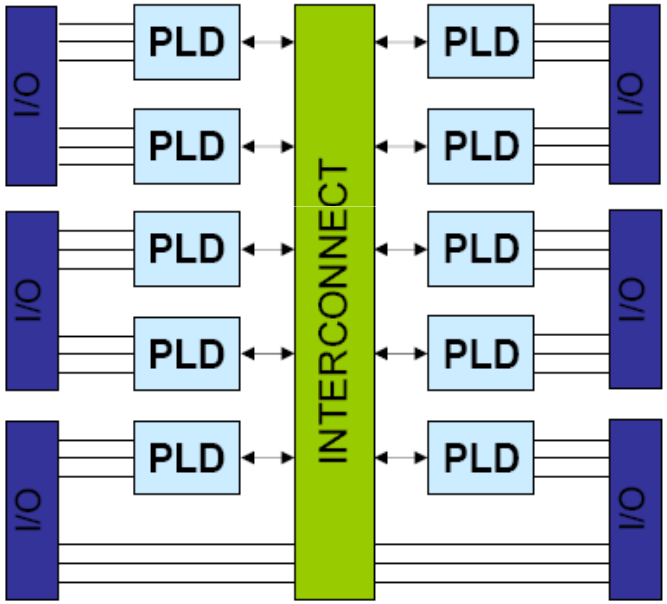


PLAN

1. Généralités sur les composants logiques programmables
2. SPLD : PAL, GAL, …
3. FPGA, CPLD : Architecture et synthèse :
    1. *Technologie et architectures*
    2. *Les CPLDs*
    3. *Les FPGAs*

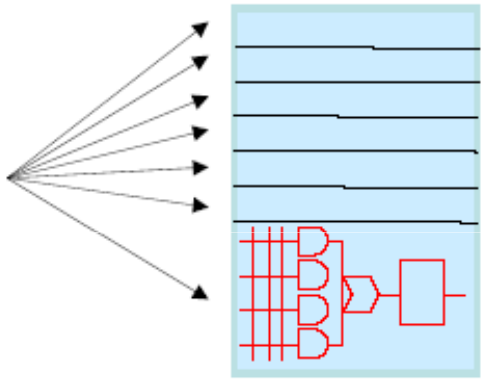
CPLD

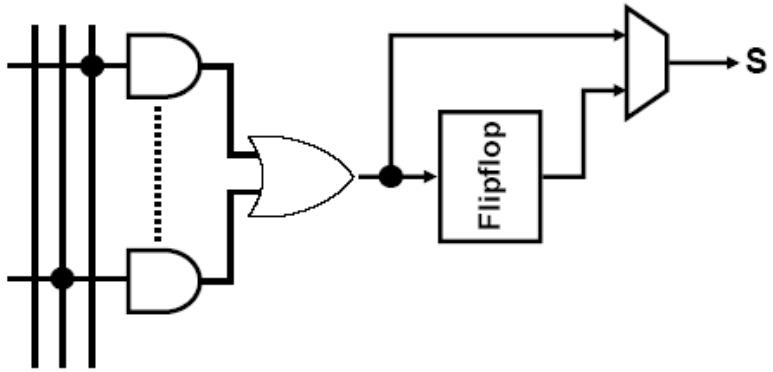
Architecture hiérarchique

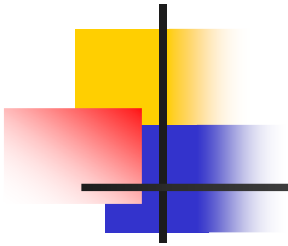


Plusieurs SPLD (par exemple PAL),
Interconnexion via une switch matrix.

1 block:
Plusieurs
Macrocells







Les CPLDs

- ALTERA : famille MAX 5000, MAX 7000, MAX 9000,
- XILINX : Famille XC 950

Famille MAX d'ALTERA (1)

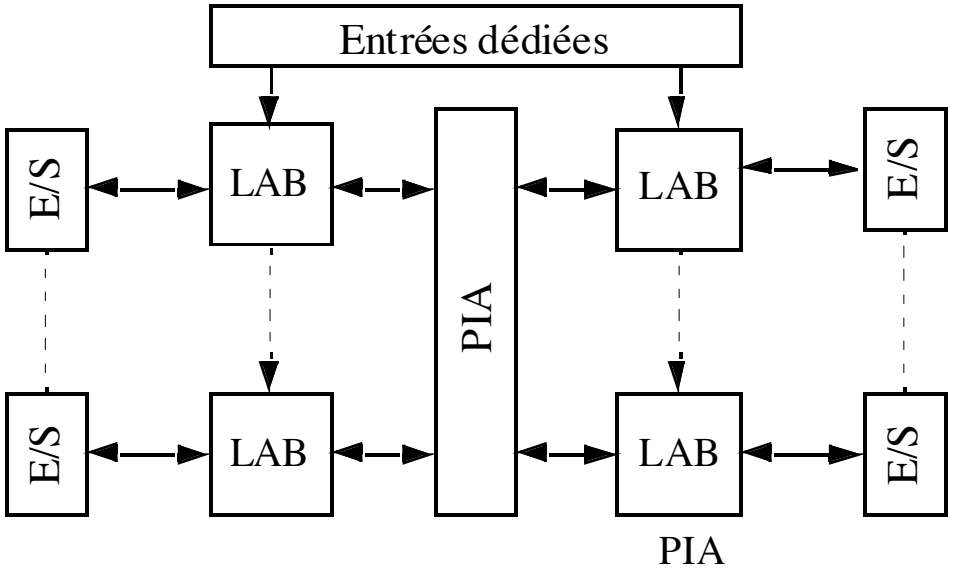
Technologie EPROM

Architecture Hierarchique ⟶ **Ensemble de PAL interconnectés entre eux;**

Ces PAL sont appelés LAB.

(Logic Array Bloc)

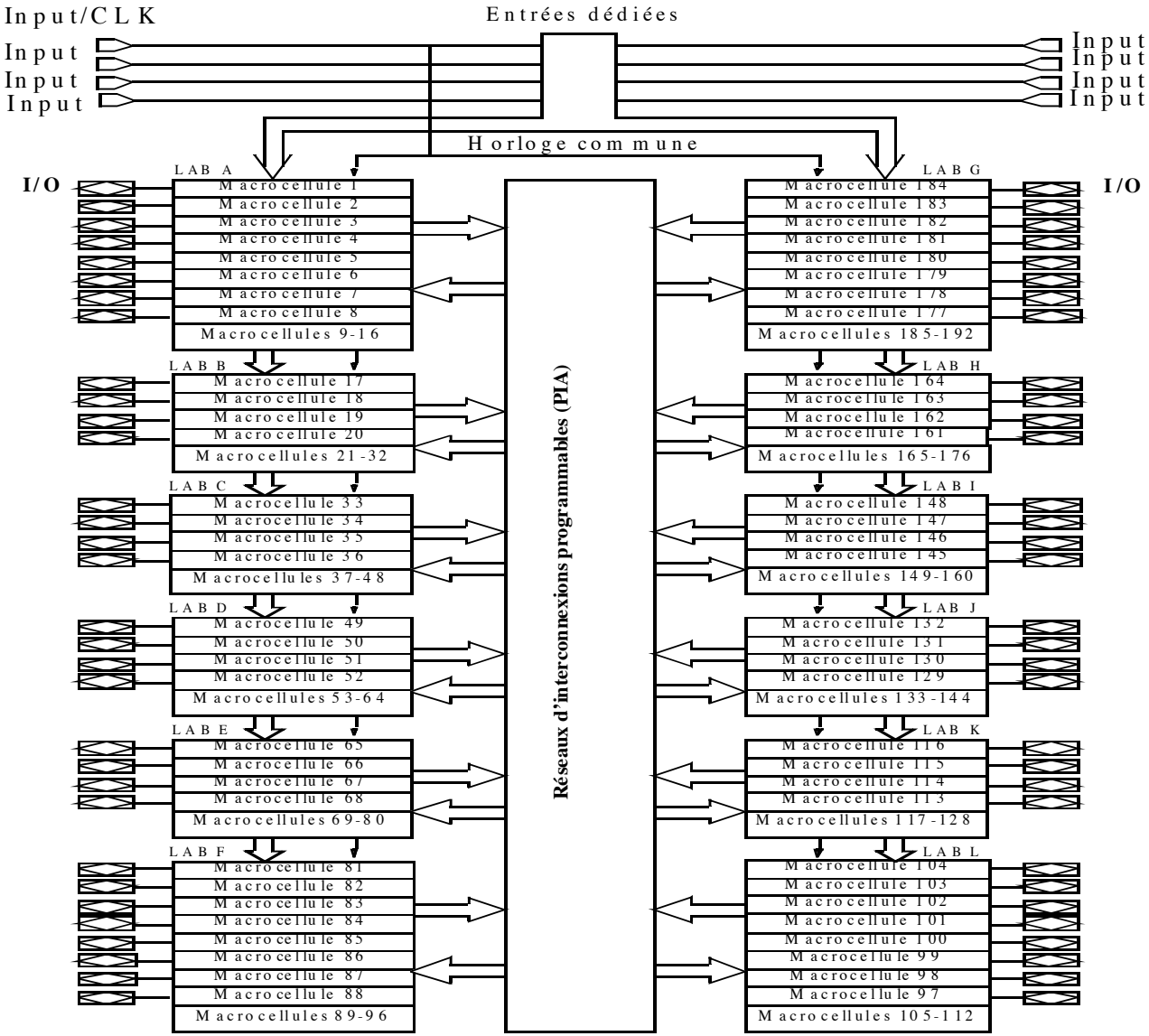
Equivalent à un PAL comportant 16 macrocellules de sortie.



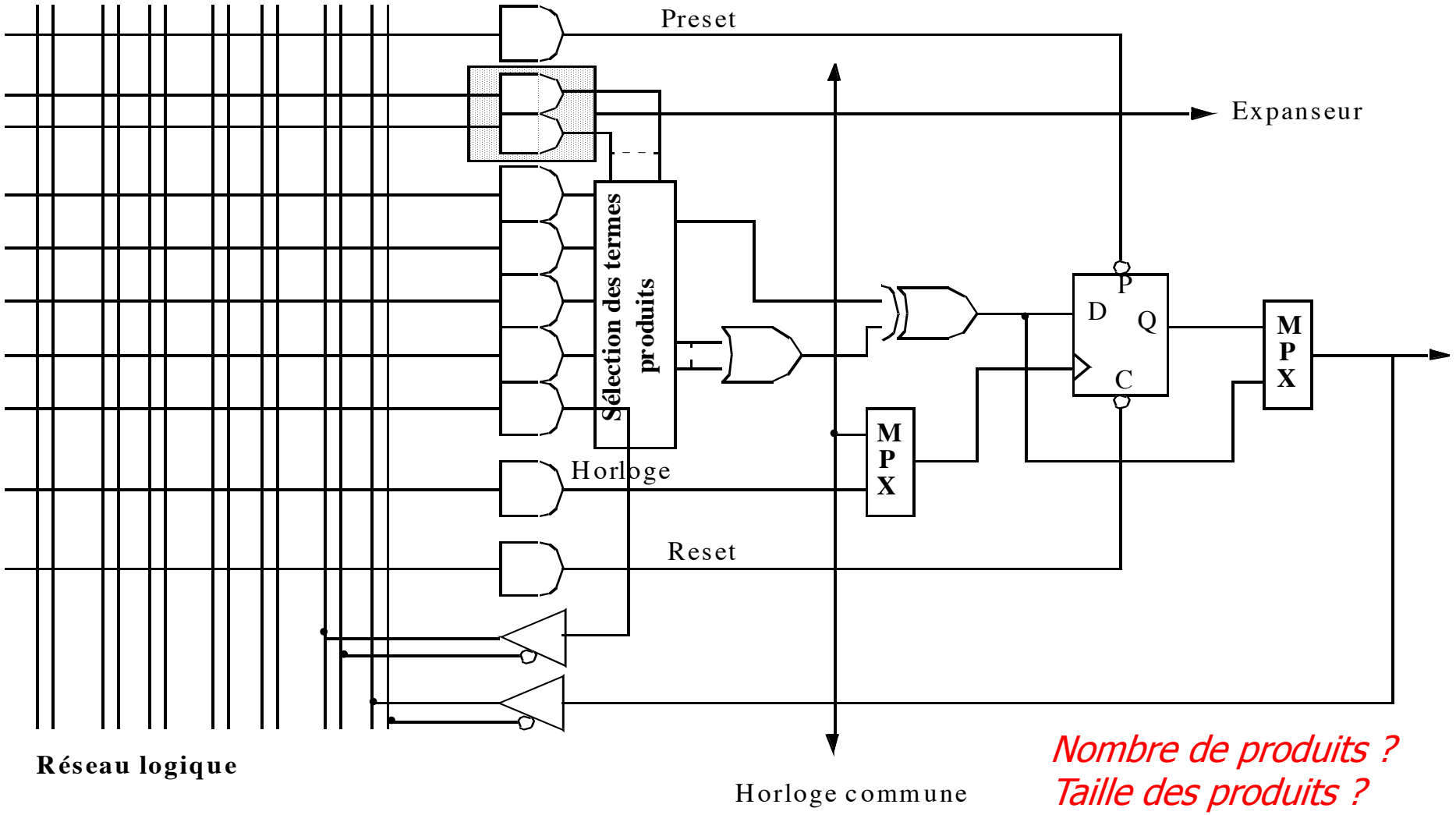
PIA

PIA : Programmable Interconnect Array
Le PIA permet de connecter n'importe entrée ou sortie de LAB vers n'importe quelle sortie ou entrée d'un autre LAB.

Exemple : Organisation interne d'un EPM 5192

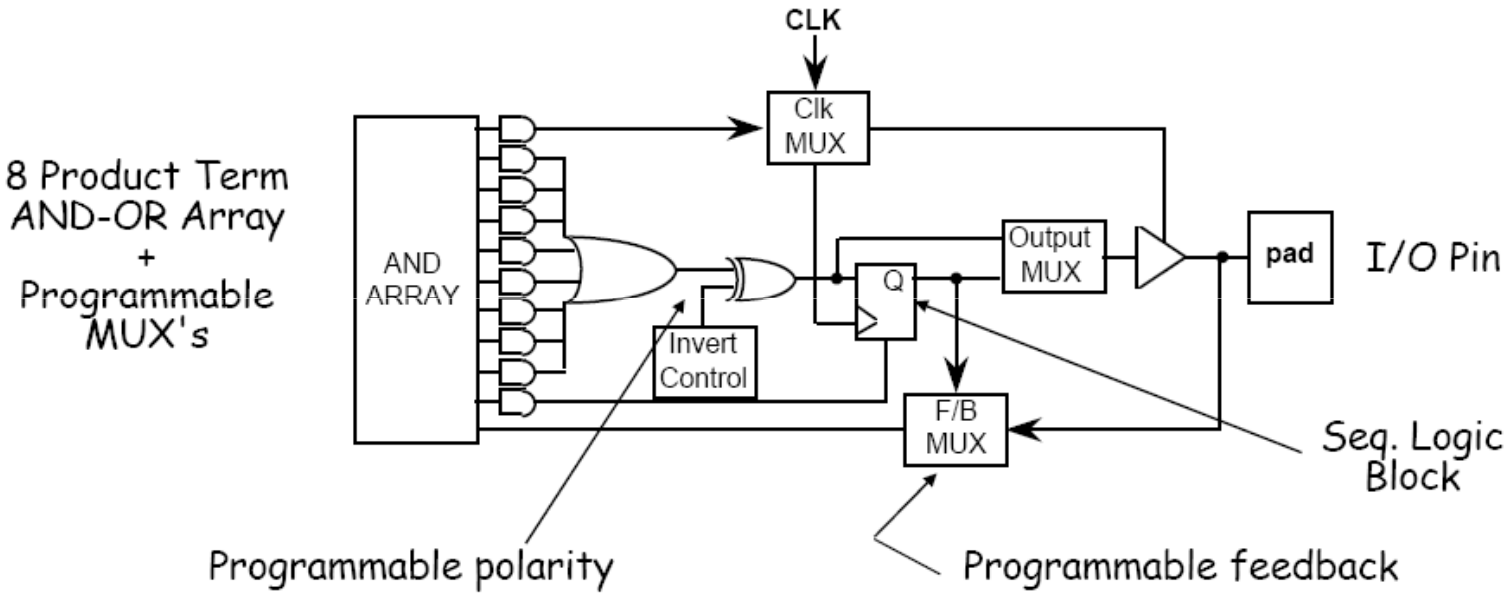


Macrocellule de sortie de la famille MAX5000



Nombre de produits ?
Taille des produits ?

Macrocellule de sortie de la famille MAX7000



Caractéristiques de la famille MAX7000B

	EPM7032	EPM7064	EPM7128	EPM7256	EPM7512
Usable Gates	600	1250	2500	5000	10000
Macrocells	32	64	128	256	512
Logic array blocks	2	4	8	16	32
Maximum User I/O Pins	36	68	100	164	212
Tpd (ns)	3,5	3,5	4	5	5,5
Tsu (ns)	2,1	2,1	2,5	3,3	3,6
Tfsu(ns)	1	1	1	1	1
Tco (ns)	2.4	2.4	2,8	3,3	3,7
Fcnt (Mhz)	303	303	243.9	188.7	163.9

Tpd : temps de réponse de réponse de la cellule (propagation delay time)
Tsu : Temps (Register Setup Time) de stabilité des données avant un front actif d'horloge (logic cell)
Tfsu : Global clock setup time for I/O cell
Tco : clock-to-output delay
Fcnt : Maximum internal global clock frequency

Add : 32 bits : 64 entrées
32 sorties.

$S_i = A_i \oplus B_i \oplus R_{i-1}$

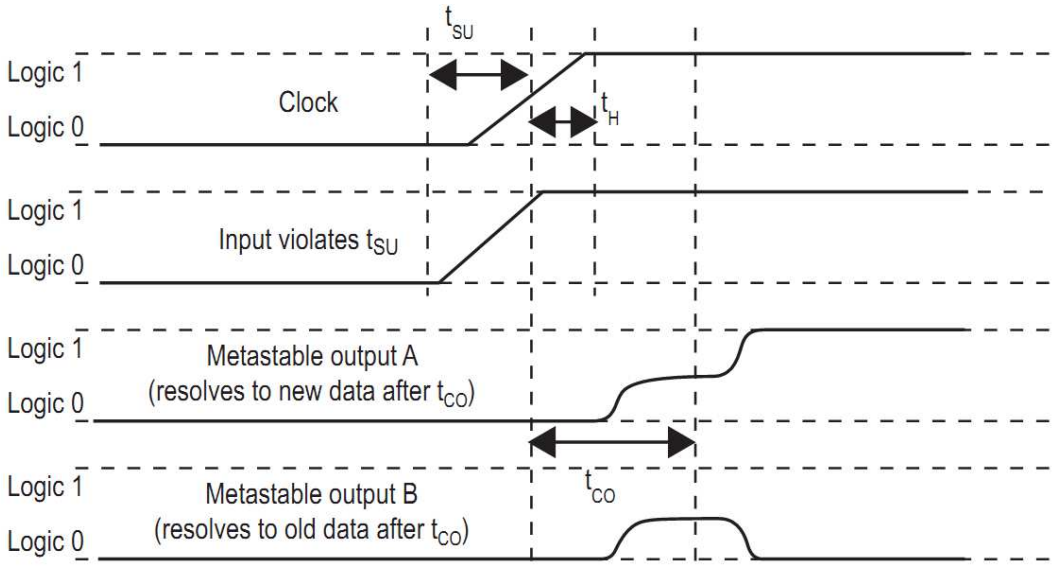
$= \overline{A_i} \cdot \overline{B_i} \cdot R_{i-1} + \overline{A_i} \cdot B_i \cdot \overline{R_{i-1}} + A_i \cdot \overline{B_i} \cdot \overline{R_{i-1}} + A_i \cdot B_i \cdot R_{i-1}$) 4 cell.

$R_i = A_i B_i + A_i R_{i-1} + B_i R_{i-1}$) 1 cell

32 bits ⇒ 32 × 2 = 64 cells.

Fréquence de fonctionnement ?

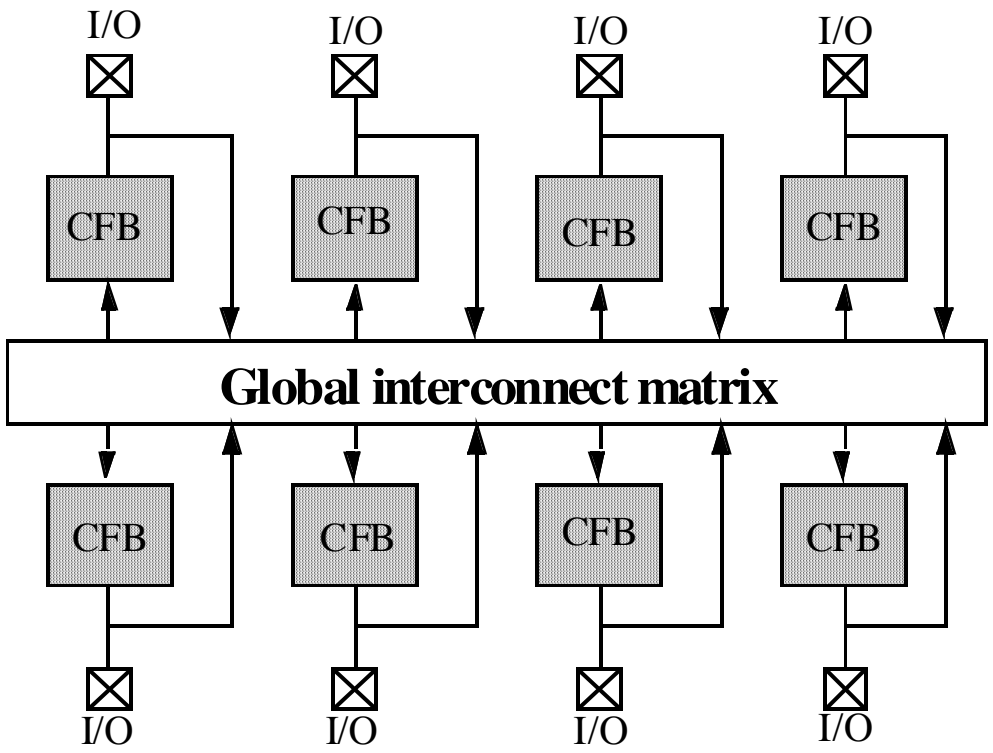
- Tsu : Temps (register Setup Time) de stabilité des données avant un front actif d'horloge
- Th : Temps de maintien des données après le front actif d'horloge (register Hold Time)
- Tco : clock-to-output delay
- mean time between failures (MTBF)



Famille Flashlogic d'ALTERA (EPX)

Technologie : SRAM

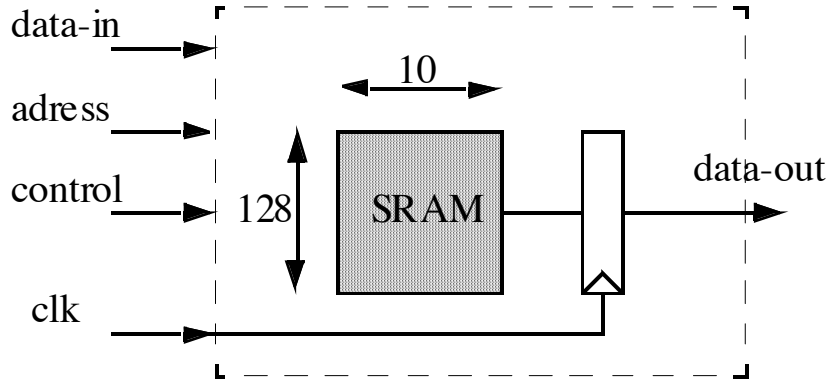
Architecture Hierarchique



CFB in PAL mode :

équivalent à un PAL24V10

CFB in SRAM mode :

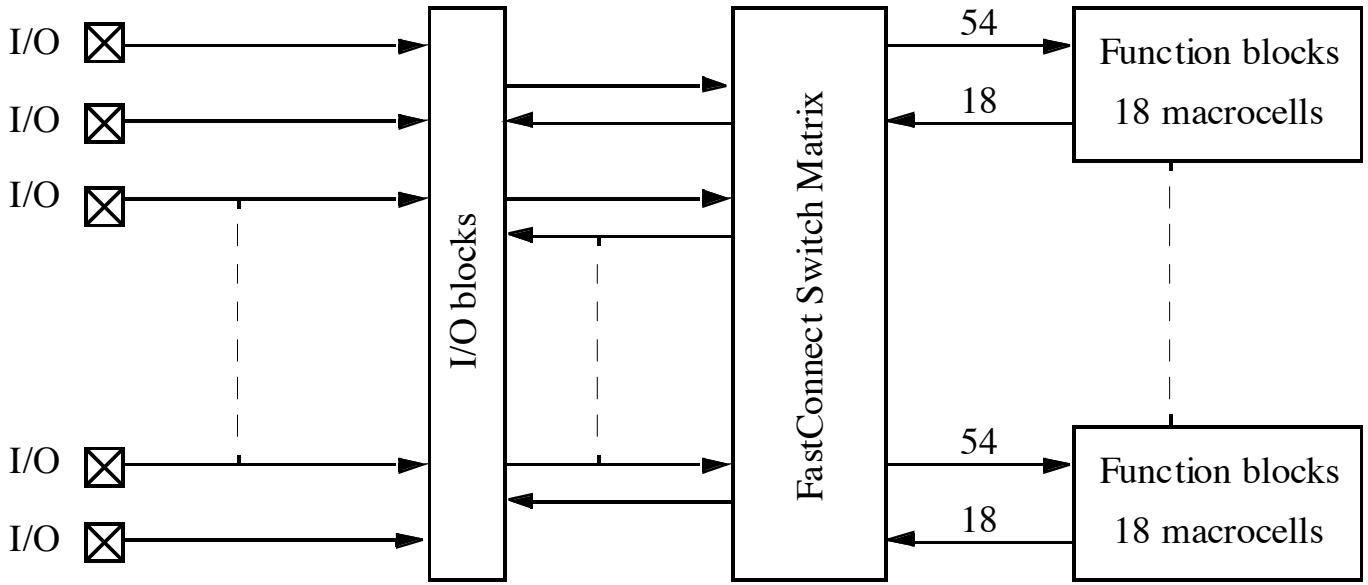


Application considérons une application vidéo.
Quel sera le gain si l'image est en interne ou en externe.

Famille Xilinx XC 950

Technologie EPROM

Architecture Hierarchique

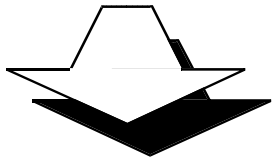


Synthèse des CPLDs

Equations logiques

↓

Déterminer le nombre de cellules



Exemple 1 : (MAX 5000)
F = abcd + adfg + abce + bdf + cdeg + aefg.
Première CE : F' = abcd + adfg + abce + bdf + cdeg,
Deuxième CE : F = F' + aefg.

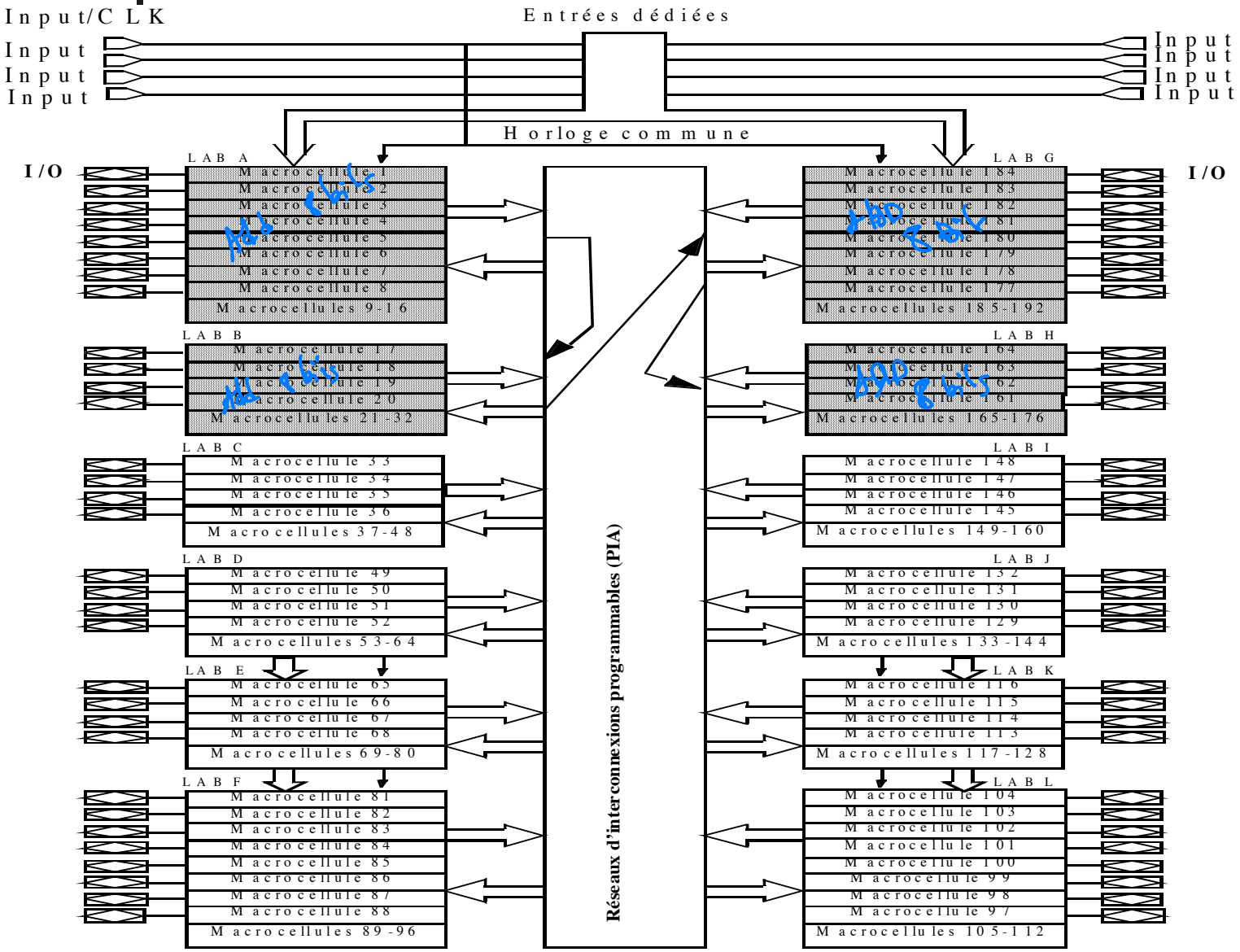
Exemple 2 : additionneur 32 bits
Si = Ai ⊕ Bi ⊕ Ri-1 et Ri = Ai.Bi + Ai.Ri-1 + Bi.Ri-1

Exemple 3 : déterminer le nombre de cellules pour une porte AND, OR, NAND, XOR à 12 entrées (Max5000 et MAX7000)

Soit N le nombre d'entrées de la porte présente dans les cellules (5 pour la famille MAX5000, 8 pour la famille MAX7000) et P le nombre de produits de la fonction Fi à réaliser.
Le nombre de cellules NC est :

Si (P < ou = à N) alors NC = 1,
Si P> N alors NC = $\lceil (P - N)/(N-1) \rceil + 1$.

Synthèse de CPLDs

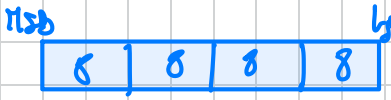


Additionneur 32 bits à retenue propagée :

- Effectuer le placement sur le CPLD,
- Etablir l'équation du temps de réponse en fonction du $T_{PD_cellule}$ et T_{PD_PIA}.
- Pour les circuits 7064 (T_{PD_PIA} = 4ns) et 7512 (T_{PD_PIA}=6,5ns), calculer ce temps de réponse.

Qu'en déduisez vous ?

64 cellules : on divise en 4 bits.



tpd / tco. (key passi bascule).

$C_{tB} = Labo.$

Max: 5000.

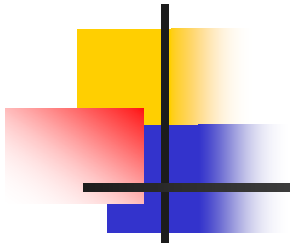
Σ Produit => 5 → 2 passages

Max 700

8 → 1 passages

16 bits $S = A.B.C + A.D.E + A.F + \bar{E}F + DE + EC.$

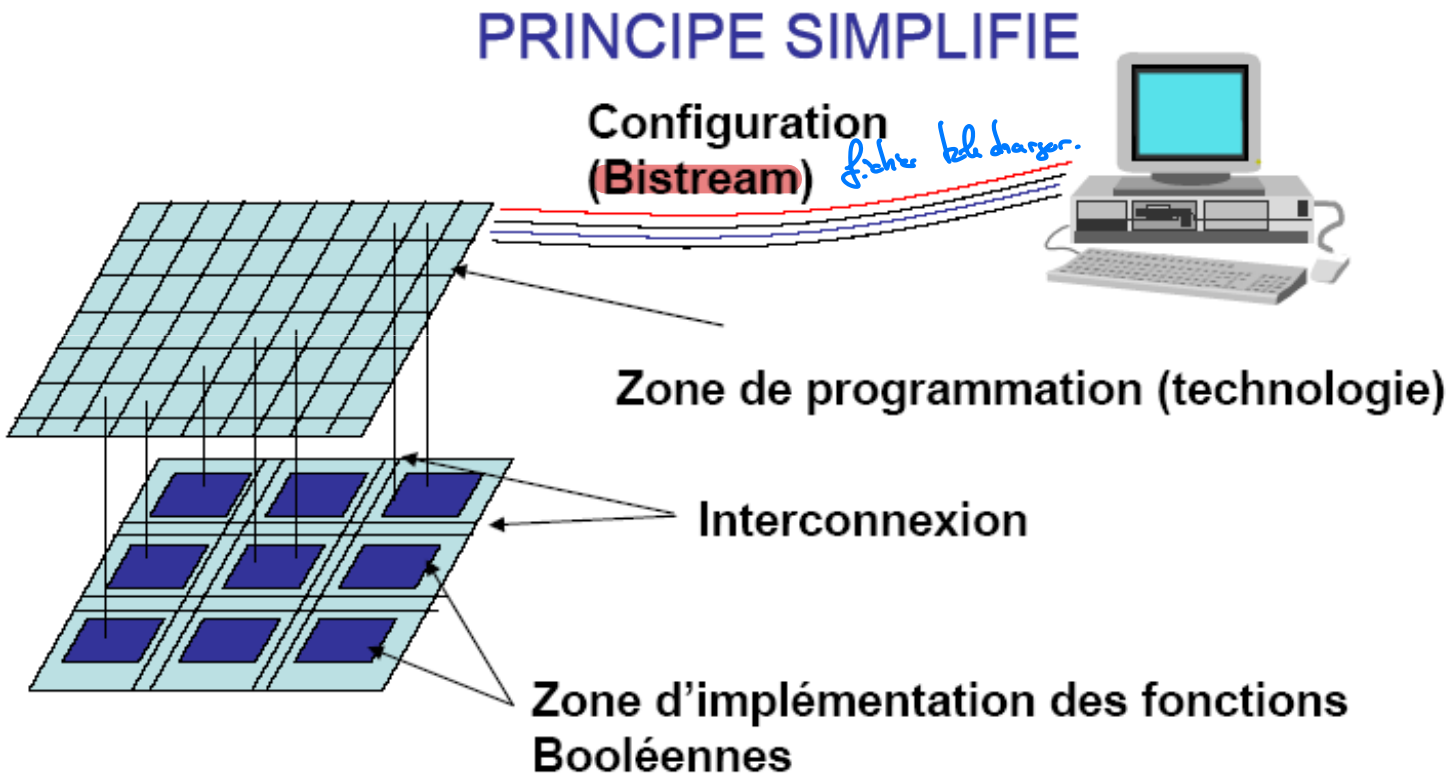
1 2 3 4 5 produits.



PLAN

1. Généralités sur les composants logiques programmables
2. SPLD : PAL, GAL, …
3. FPGA, CPLD : Architecture et synthèse :
    1. *Technologie et architectures*
    2. *Les CPLDs*
    3. *Les FPGAs*

Technologie de programmation : principe

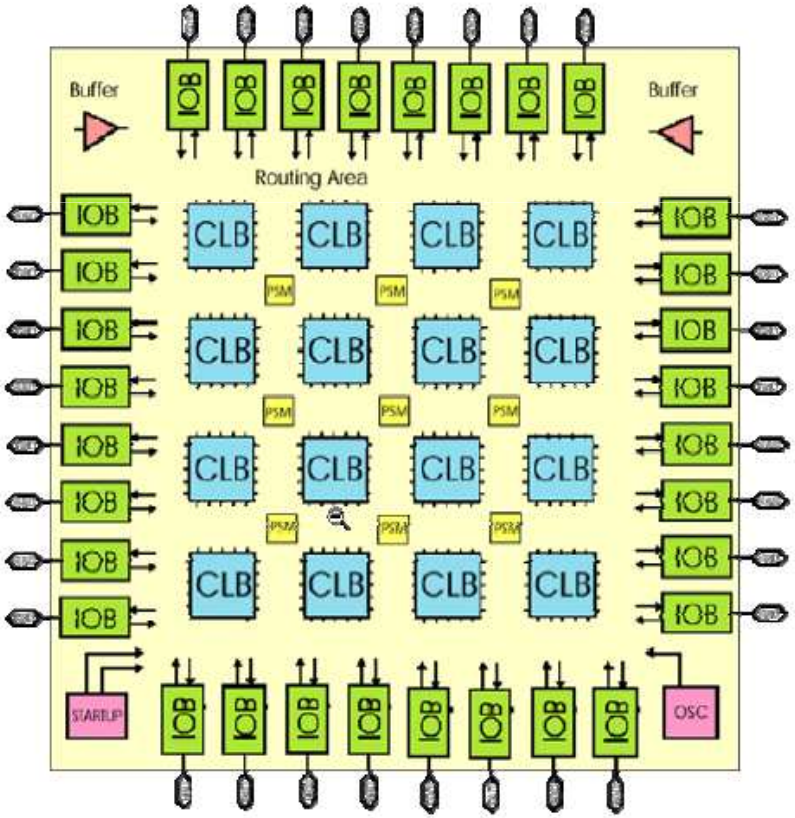


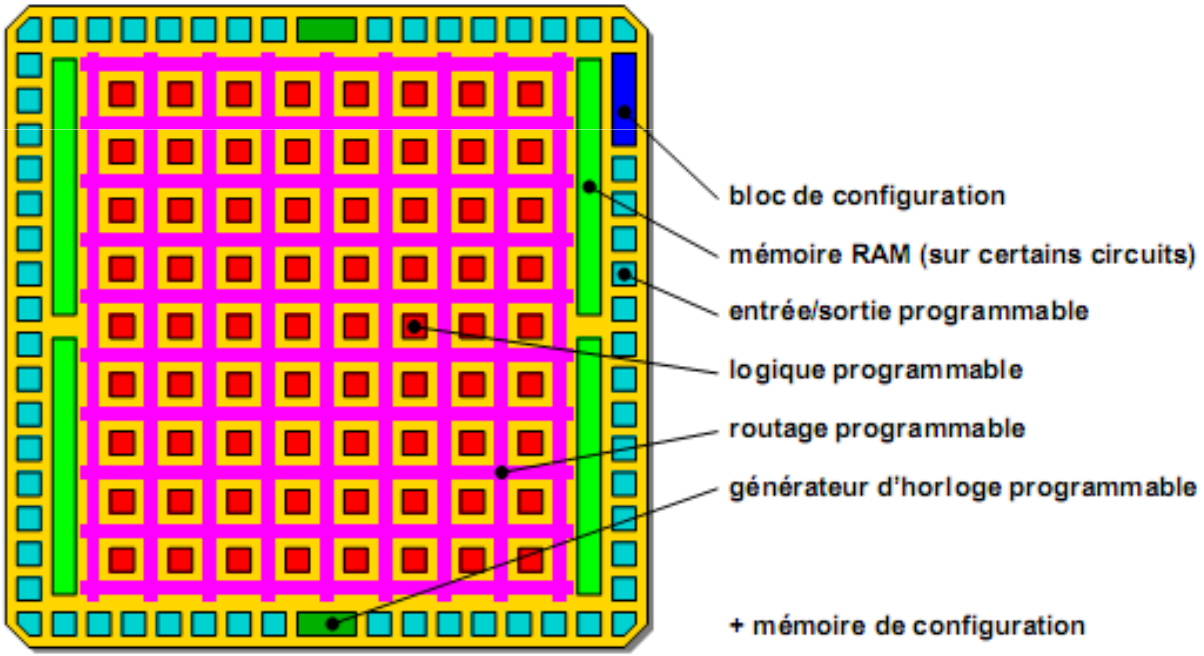
Structure générale d'un FPGA-MESH

Matrice de blocs reconfigurables

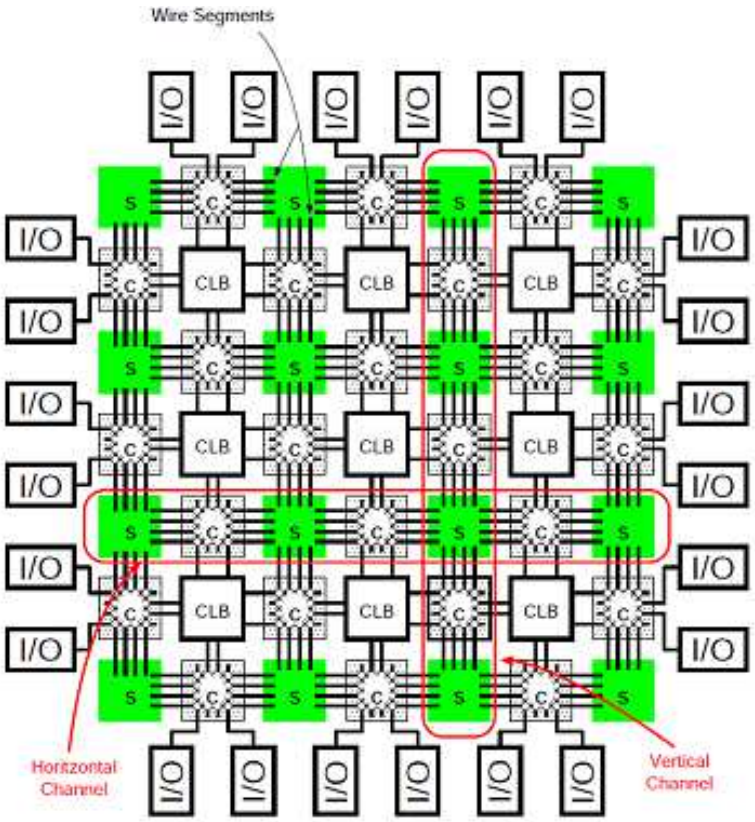
Interconnexion reconfigurable

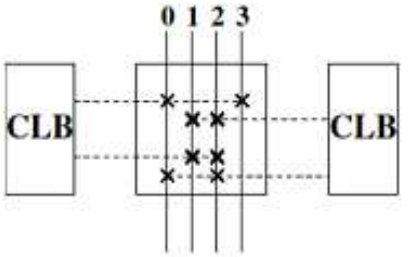
CLB : élément logique + élément séquentiel



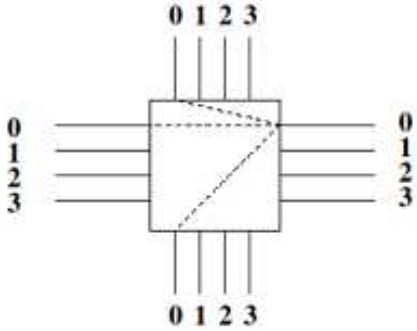


Autre exemple de configuration





Bloc de connexion (c)

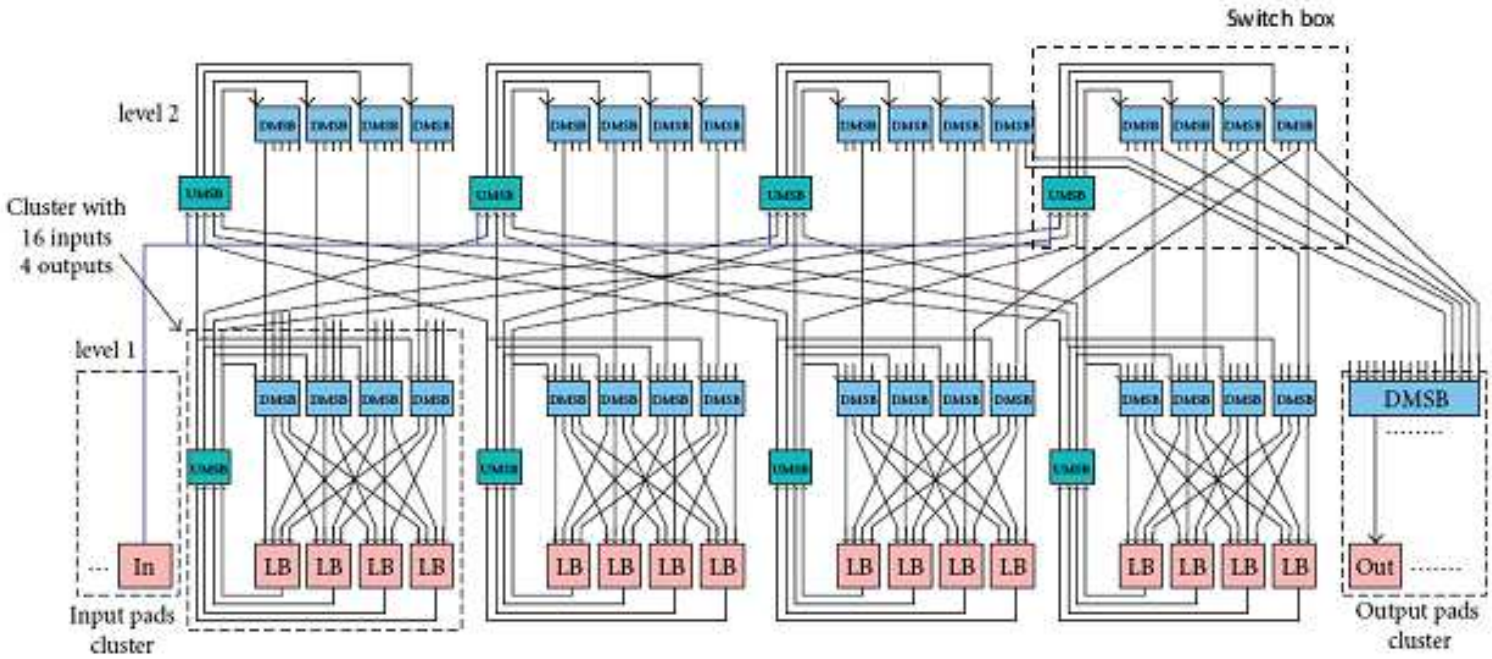


Switch box (s)

=> Inconvénient une grande partie de la surface est due aux communications

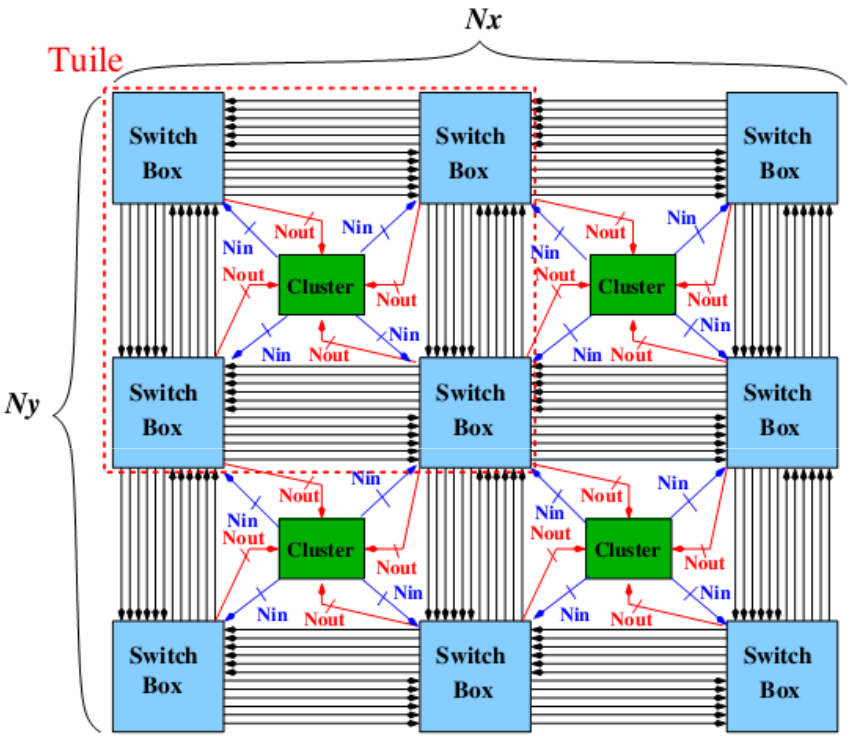
Autre architecture FPGA Tree_based

Exemple : Tree_based FPGA (Altera, xilinx)

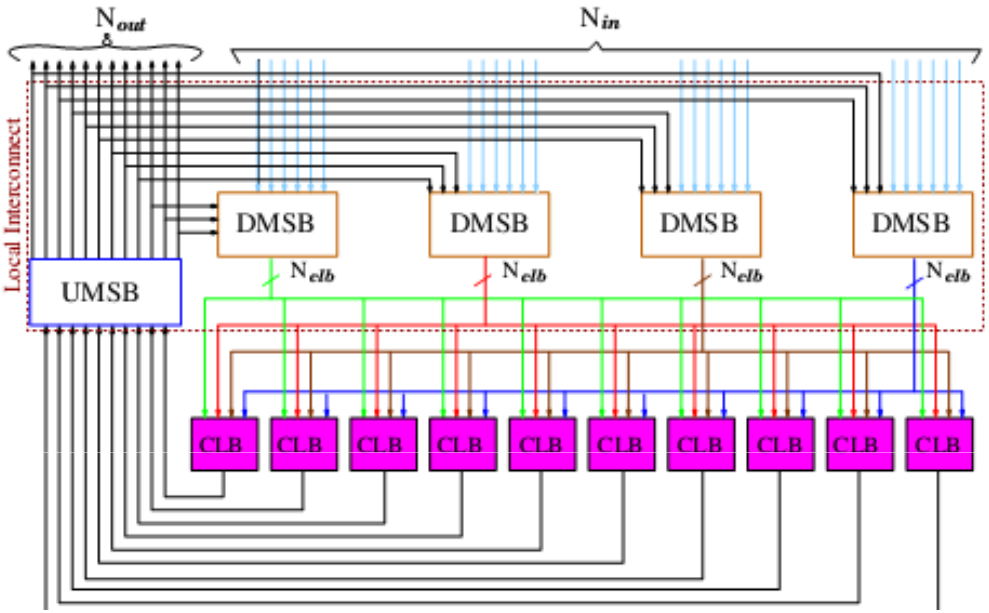


Réduction de l'interconnexion
Augmentation possible du temps des communications
Problème de scalabilité

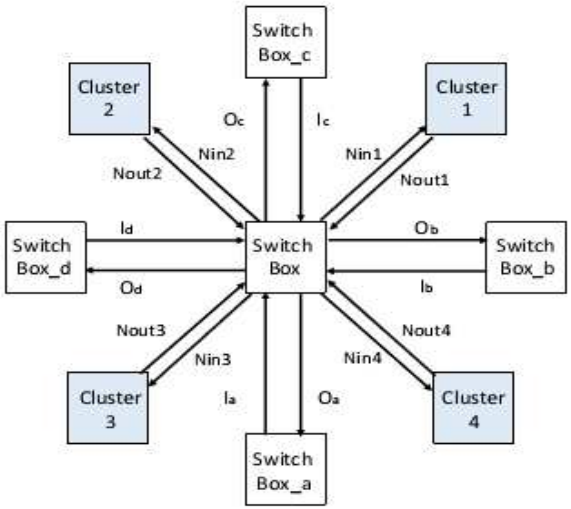
Archietcture FPGA – MESH of clusters



Réseau d'interconnexion



Architecture d'un cluster



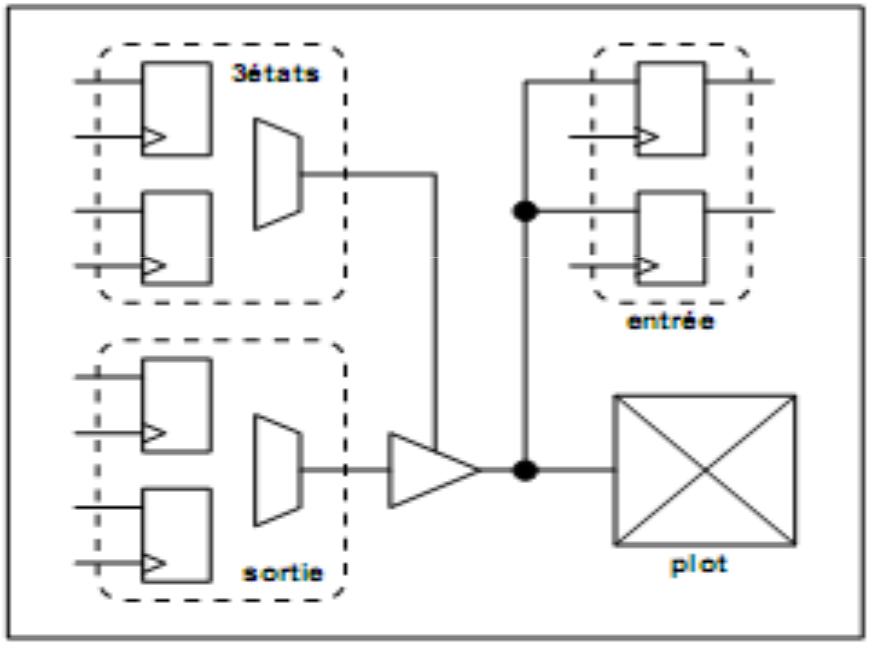
Blocs d'entrée/sortie programmables

- Les blocs d'entrée/sortie servent d'interface entre les pattes (plots) du circuit et le coeur du FPGA (via le routage programmable).
- Il y a différents types d'entrées/sorties :
  - entrées/sorties utilisateur

    adaptation des signaux (CMOS, TTL, 3.3, 2.5, 1.8, 1.5, paire di.)

    Baisse constante de la tension d'alimentation : 1998 ->350 nm,3.3 V; 2000-> 180 nm, 1.8 V; 2001 -> 150 nm, 1.5 V; 2003->130 nm, 1.2 V
  - alimentations séparées en zones : coeur, bancs de plots, zones basse-consommation
  - signaux d'horloge : possibilité d'avoir plusieurs horloges
  - signaux de configuration : programmation du FPGA
  - signaux de test : boundary scan JTAG pour debug

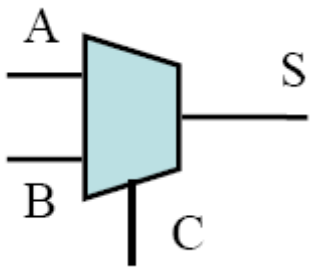
Exemple de bloc d'entrée/sortie (Virtex II)



Déterminer les fonctionnalités de cette cellule d'I/O.

1 entrée : synchroniser

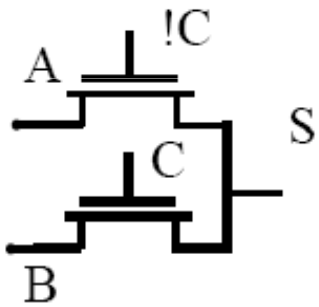
Cellule de base : le multiplexeur



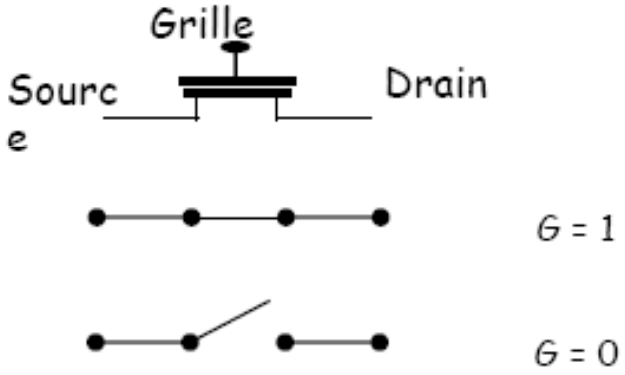
Si C=0, S=A

Si C=1, S=B

$S = \bar{C}.A + C.B.$



Rappel sur le fonctionnement du transistor

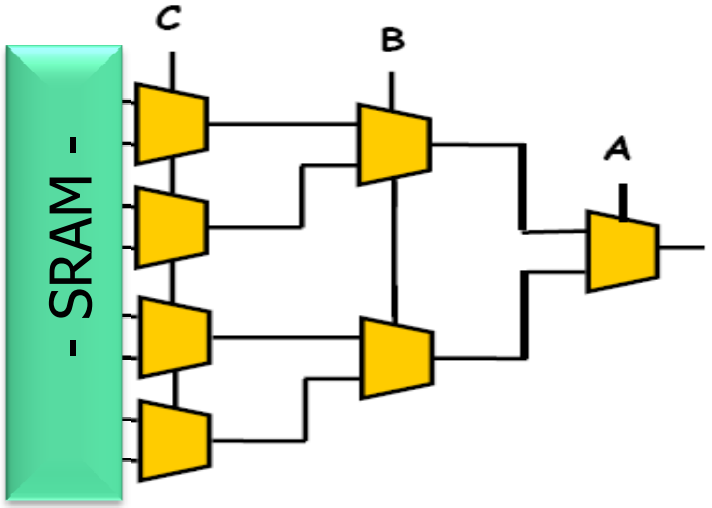


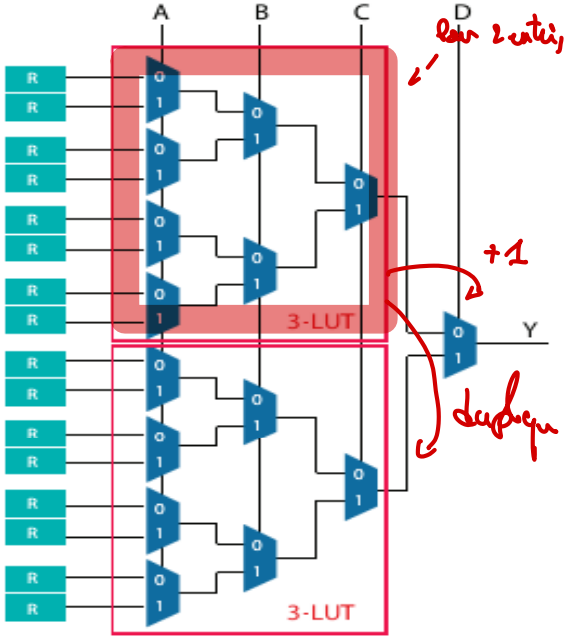
Transistor NMOS = commutateur

Peut-on générer n'importe fonction et si oui sur quelle(s) variable (s) ? A, B, C

Les cellules de base : avec LUT

LUT (Look Up Table) à k entrées (K-LUT) :
Permet de réaliser n'importe quelle fonction à k entrées

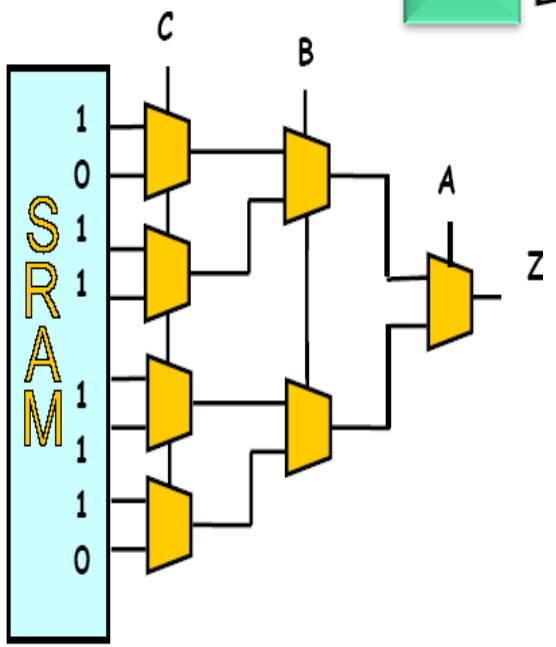




A	B	C	Z
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

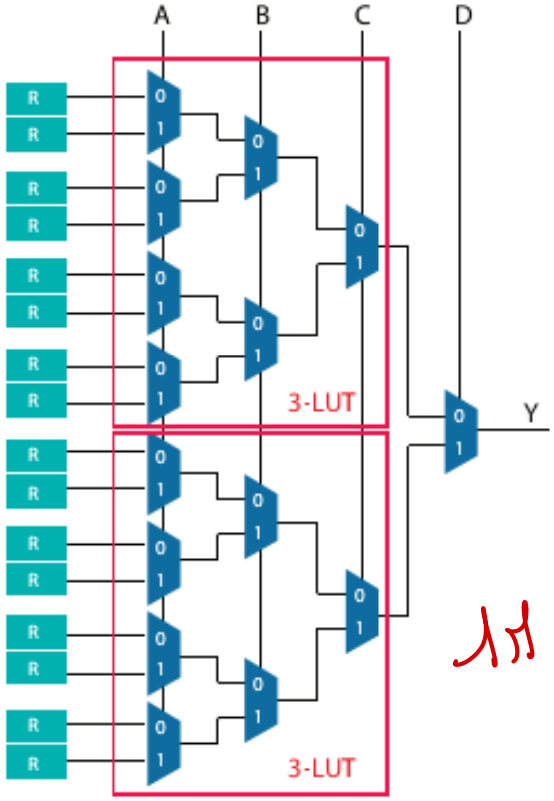
« Look Up Table »

LUT 3



LUT à 4 entrées, 6 entrées :
- Calculer le nombre de transistors équivalents pour ces cellules (4-LUT, 6-LUT).
- Combien peut-on mettre de cellules dans un circuit qui contiendrait 1M de transistors ?

LUT hiérarchisée

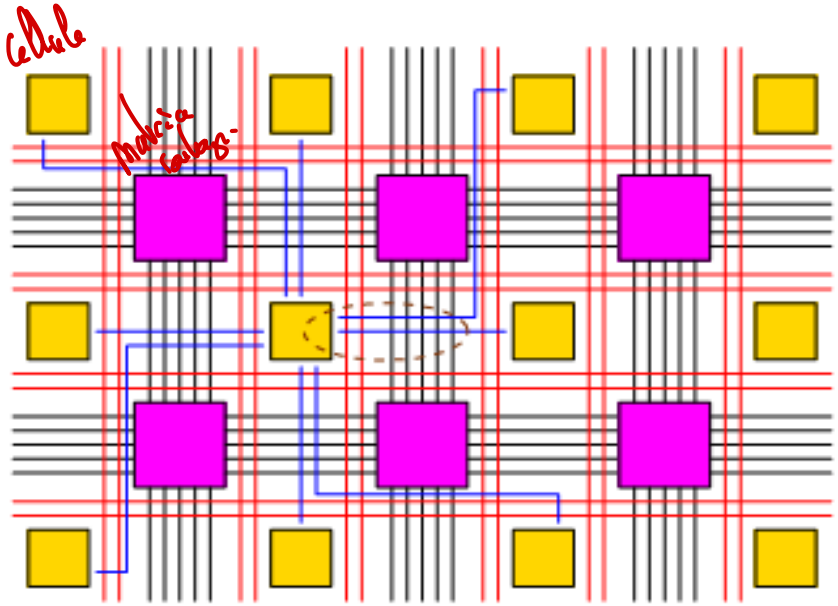


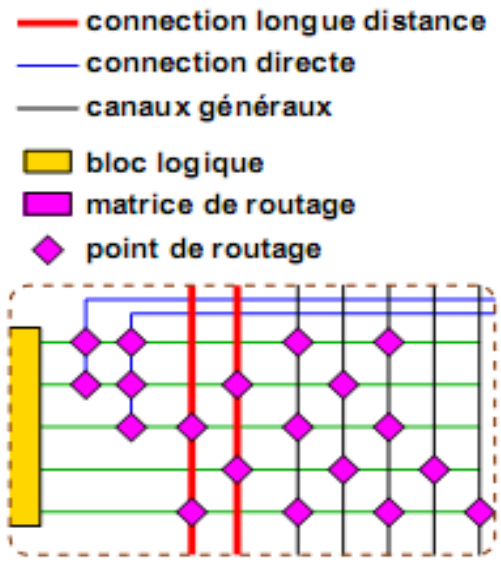
Déterminer pour une LUT à 4 entrées, un bloc logique comportant 10 cellules élémentaires, le nombre de cellules mémoires.

Routage programmable

Pour connecter les blocs logiques entre eux et les entrées/sorties, les FPGA disposent de différentes ressources de routage programmables :

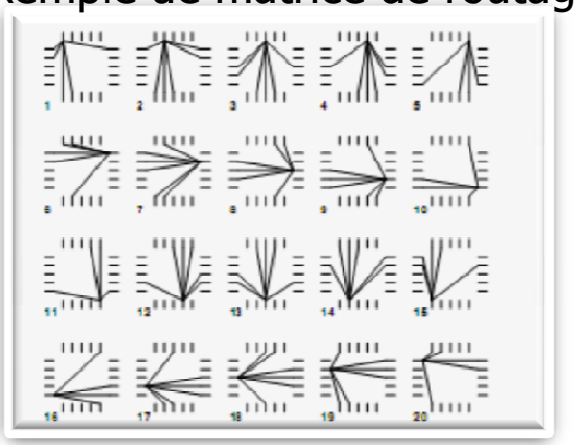
- connections directes vers les voisins proches
- connections générales à travers des matrices de routages et des canaux disposés suivant une topologie simple (grille, tore)
- connections à longue distance
- distribution d'horloge spécifique (sur des canaux dédiés)





Remarque : attention échelle fausse, blocs logiques environ 5% du circuit

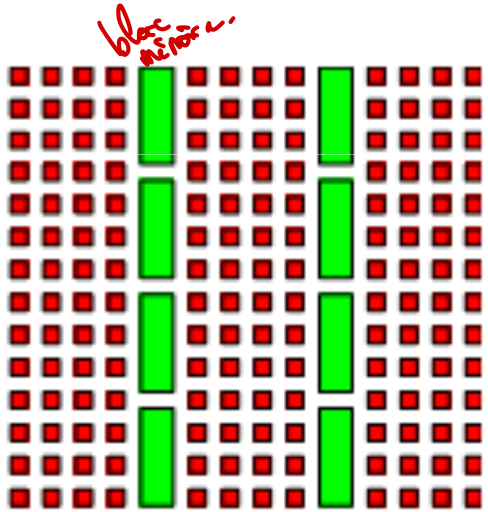
Exemple de matrice de routage :



Mémoire utilisateur

On peut utiliser les blocs logiques pour faire des petites mémoires (distributed memory) => pas très efficace si besoin important.

Introduction de blocs mémoire complets.



Cas des Virtex II : bloc de 18Kb simple ou double port, configurable en 16K x 1, 8K x 2, 4K x 4, 1K x 18, 2K x 9, 512 x 36. Il y a entre 4 (XC2V40) et 168 (XC2V8000) blocs.

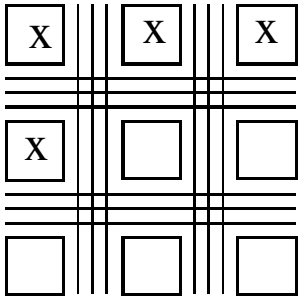
Les différentes étapes de synthèse pour les CPLDs et les FPGAs

Equations logiques

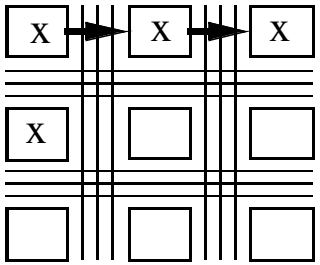
Choix du composant

Déterminer le nombre de cellules

F1 : 1 cellule
F2 : 3 cellules



Affectation des cellules (Placement)



Routage

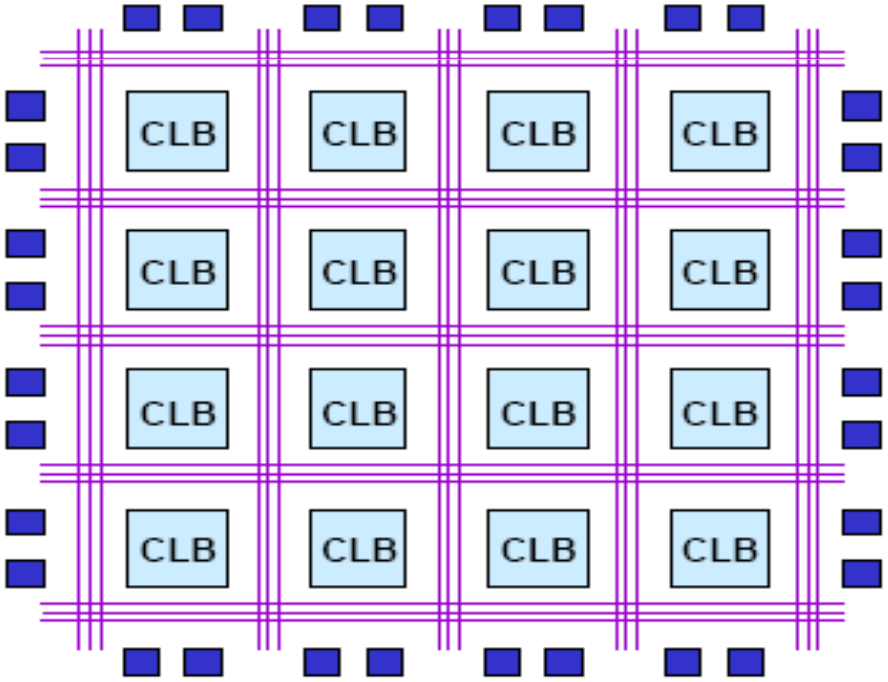
Programmation (bitstream)

QQ FPGAs

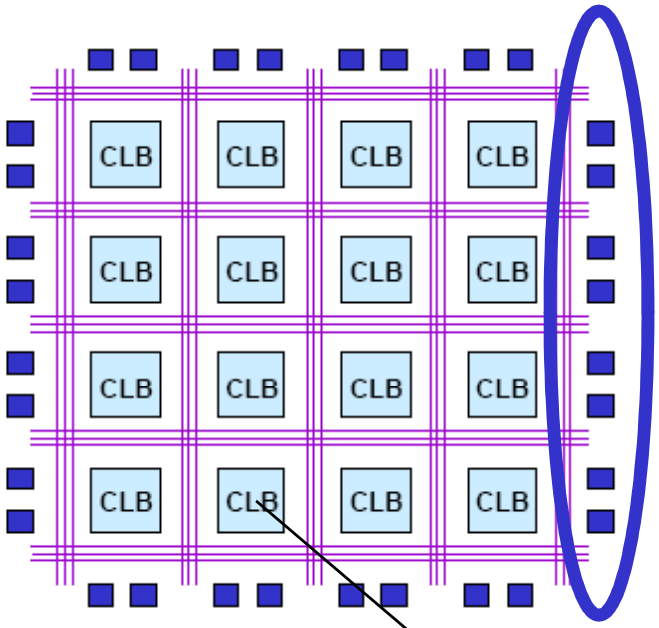
- **Cellules avec LUTs:**
  - **XILINX : famille XC 2000, XC3000, XC4000**
  - **ALTERA : Famille FLex,**
- **Cellules avec MPX**
  - **ACTEL : famille Act1, 1, 3**

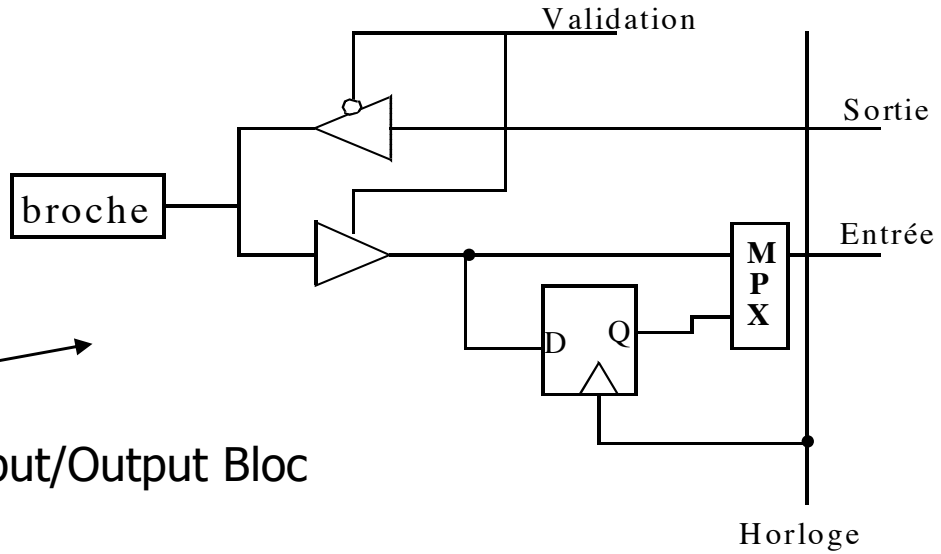
Architecture Xilinx (1) : la famille XC2000

Tableau à deux dimensions de blocs programmables appelés Configurable Logic Bloc (CLB), Canaux horizontaux entre les lignes de blocs, Canaux verticaux entre les colonnes.



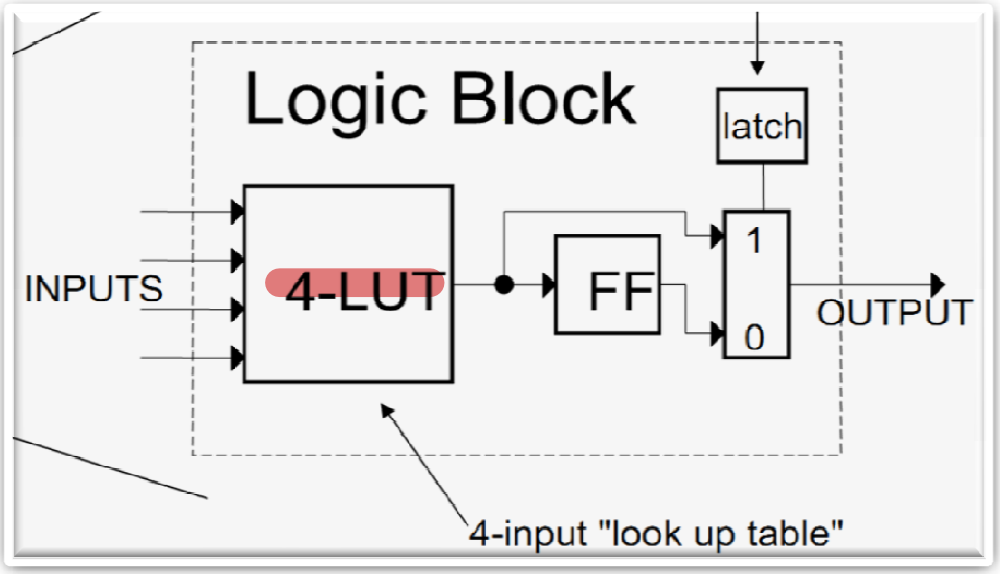
Architecture Xilinx (1) : la famille XC2000



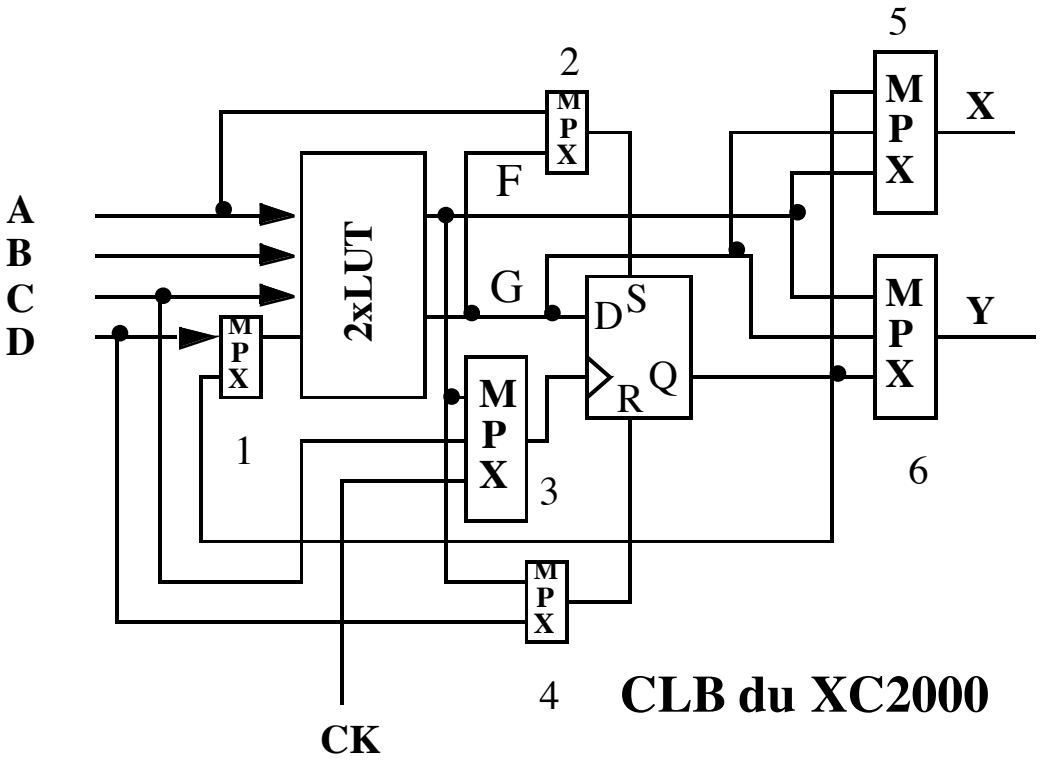


IOB : Input/Output Bloc

Exemple de bloc logique (générique)



Architecture Xilinx (2) : la famille XC2000



CLB du XC2000

Modes de fonctionnement du CLB :

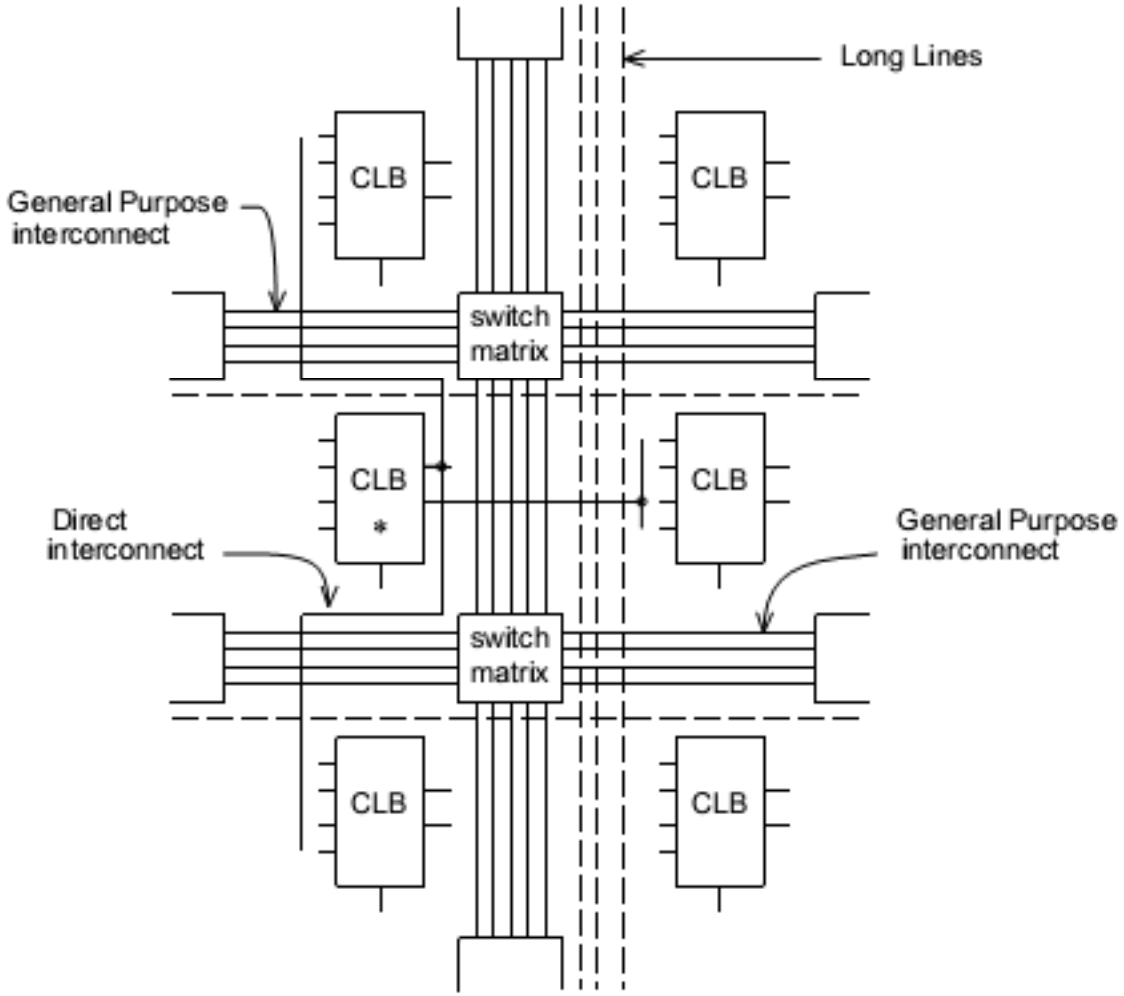
- en logique combinatoire ?

- en logique séquentielle ?

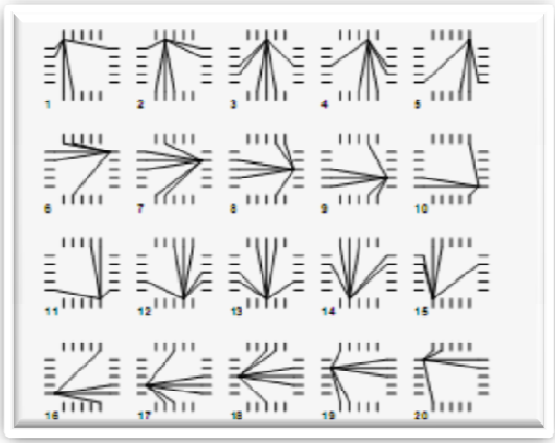
4 entrées → 2 sorties :
Soit 1 combinatoire + 1 bascule
2 combinatoire

LUT : Look Up Table à 4 entrées

Architecture Xilinx (5) : principe de l'interconnexion

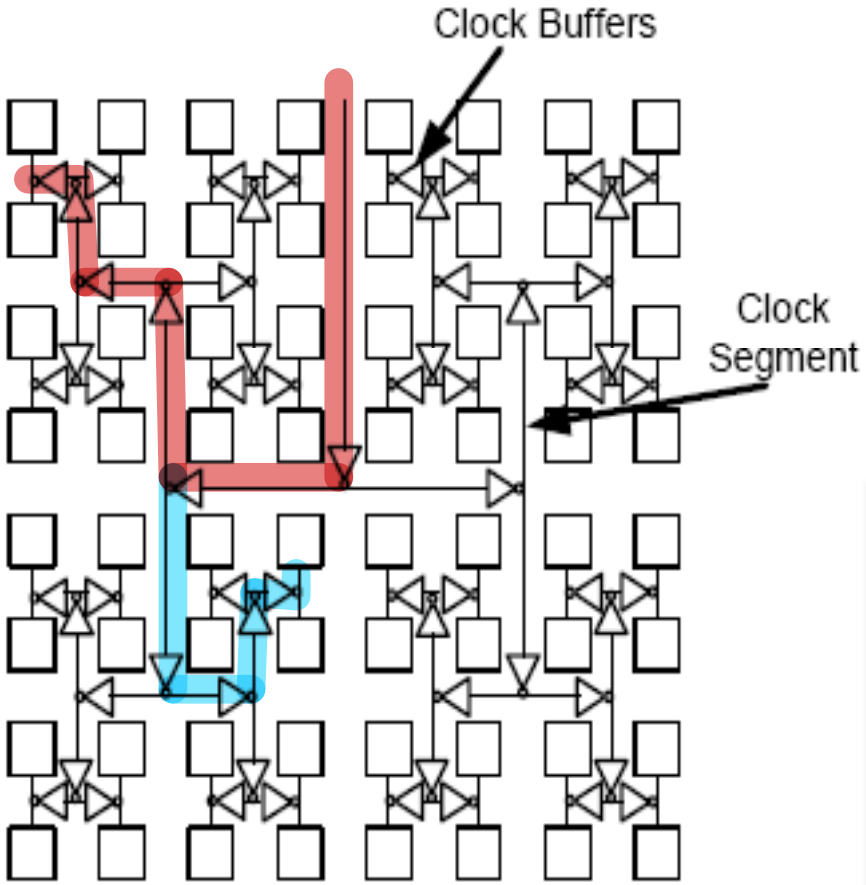


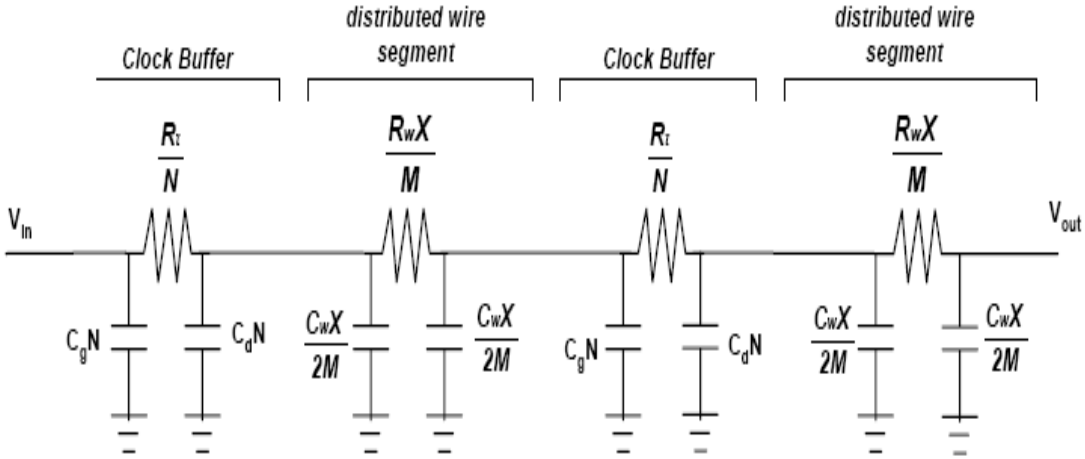
Exemple de switch matrix :



Architecture Xilinx : Distribution de l'horloge (H)

arrive en même temps.



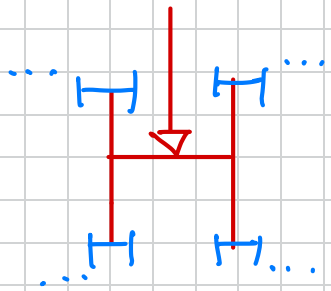


Si Tbuf est le temps de réponse d'un inverseur, indiquer le décalage entre le signal d'horloge initial et celui qui arrive sur les cellules logiques.

Exprimer les contraintes qui en résulte sur la fréquence de fonctionnement ?

Que se passe-t-il si le nombre de cellules augmentent beaucoup ?

Distribution en H.

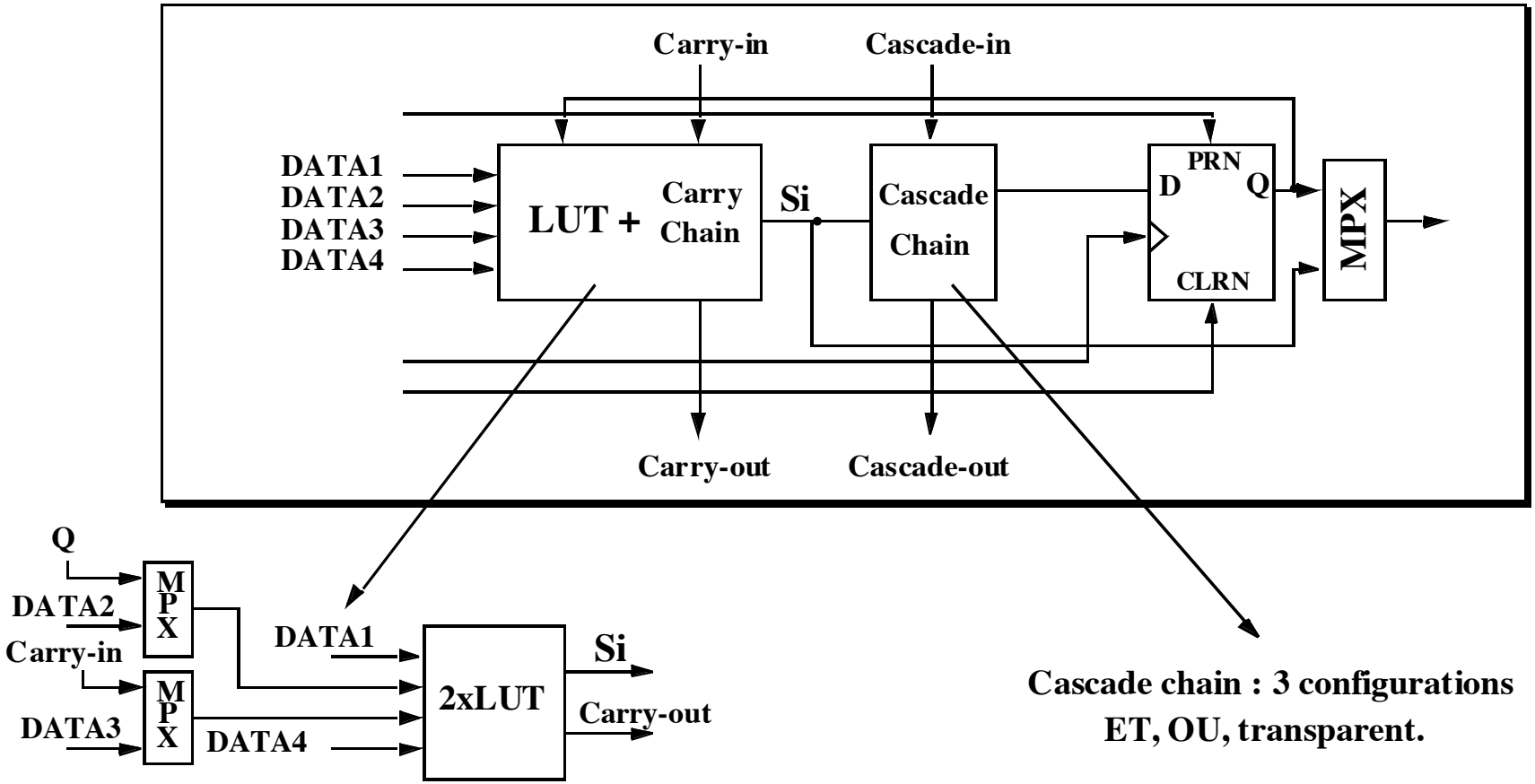


Les FPGAs

- **Cellules avec LUTs:**
  - **XILINX : famille XC 2000, XC3000, XC4000**
  - **ALTERA : Famille FLex,**
- **Cellules avec MPX**
  - **ACTEL : famille Act1, 1, 3**

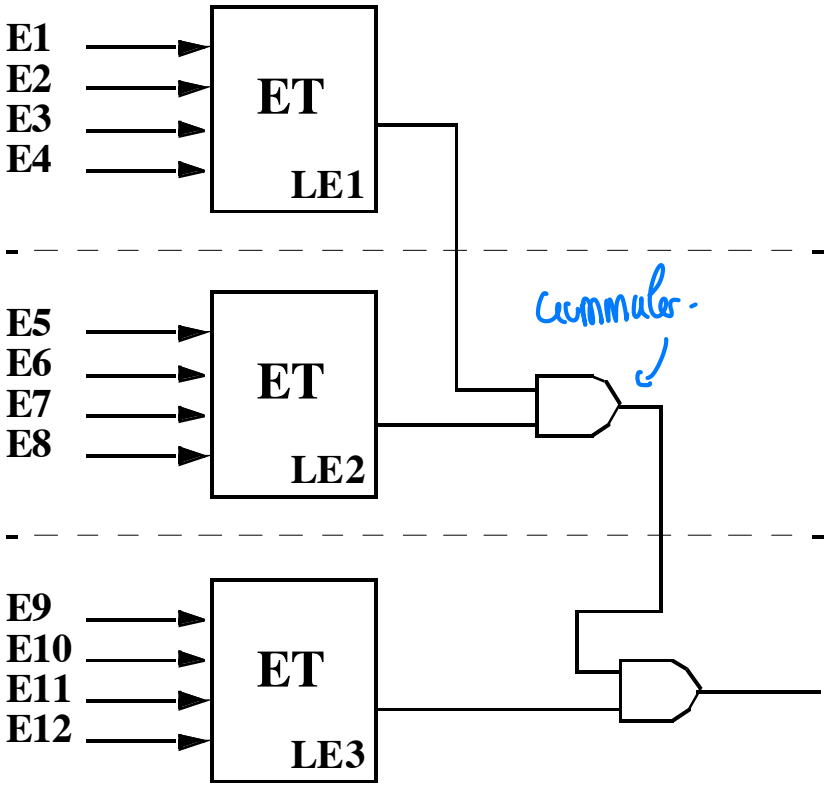
ALTERA : La Famille FLEX (Technologie SRAM)

Cellule de sortie : LE (Logic Element)

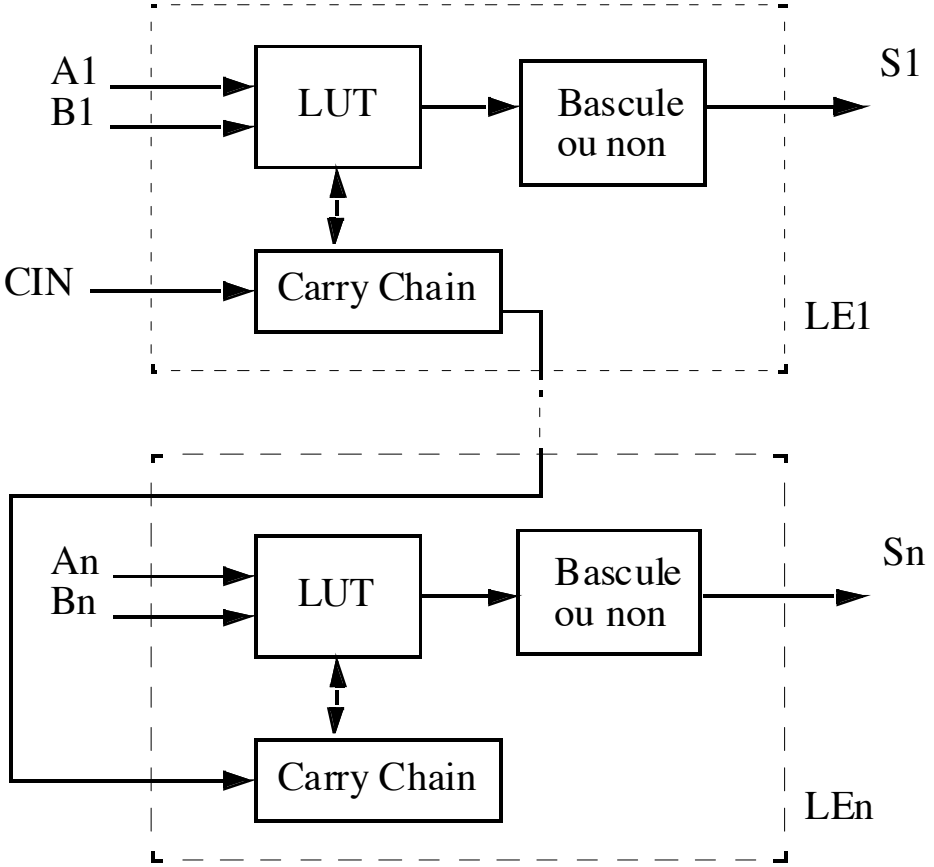


Nombre d'entrées disponibles pour les LUTs en fonction du mode de fonctionnement ?

ALTERA : La Famille FLEX (Technologie SRAM)

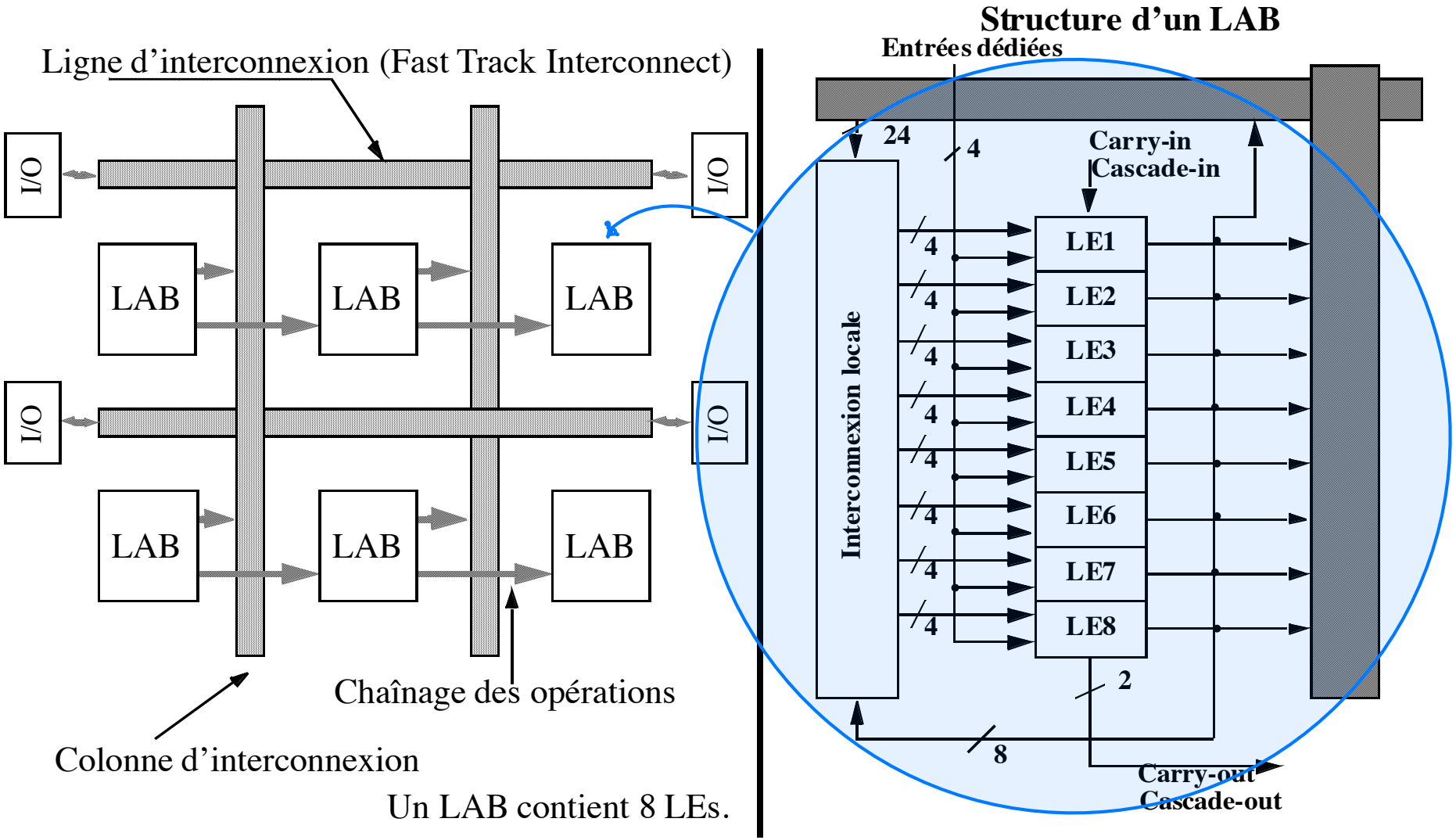


Exemple d'opérations cascadées :
ET à 12 entrées



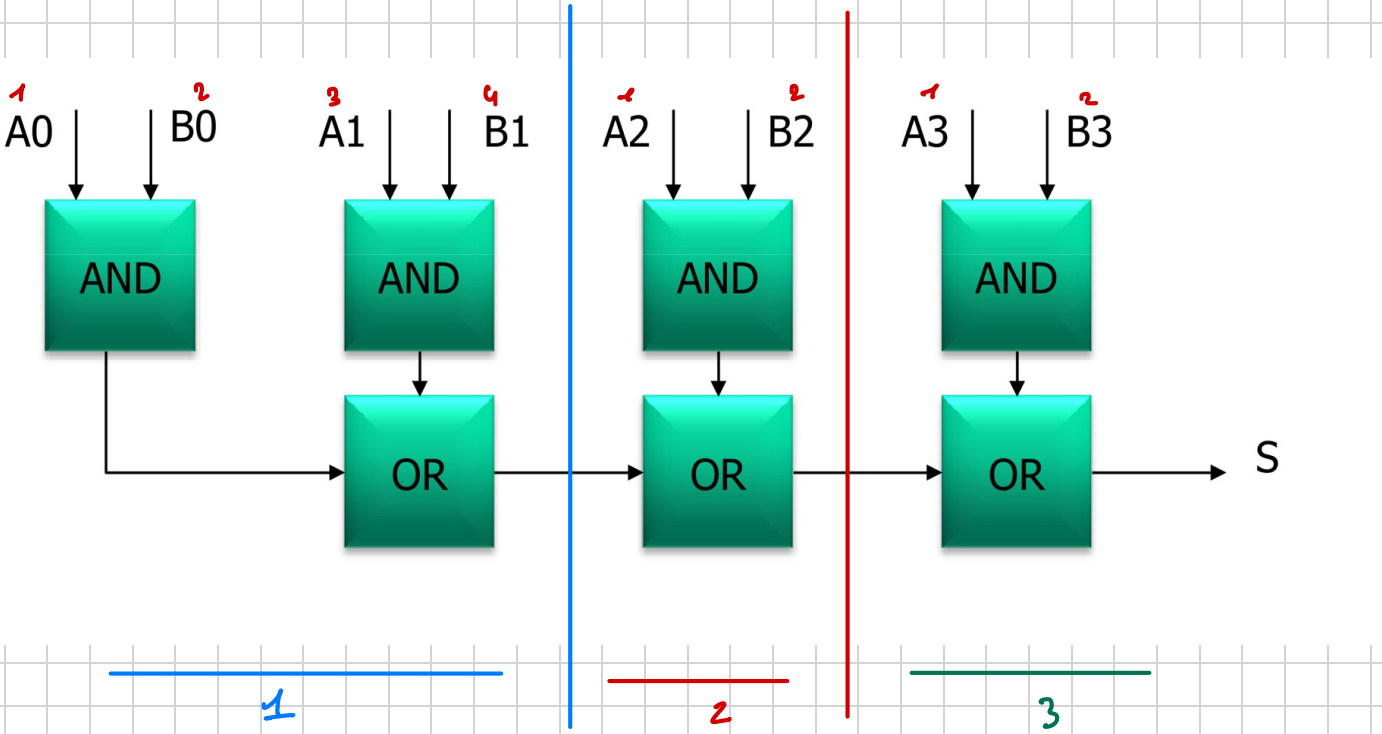
Exemple de chaînage de carry :
additionneur, soustracteur, compteur…

ALTERA : La Famille FLEX (Technologie SRAM)



k - LUT : n'importe quelle fonction à k entrée : $\sum_{6}^{4}$) entrée.

on prend un LUT 5 entrée
↳ 3 cellules



$F_1 = A_0 \cdot b_0 + A_1 B_1$

$F_2 = F_2 + A_2 B_2$

$S = F_2 + A_3 B_3$



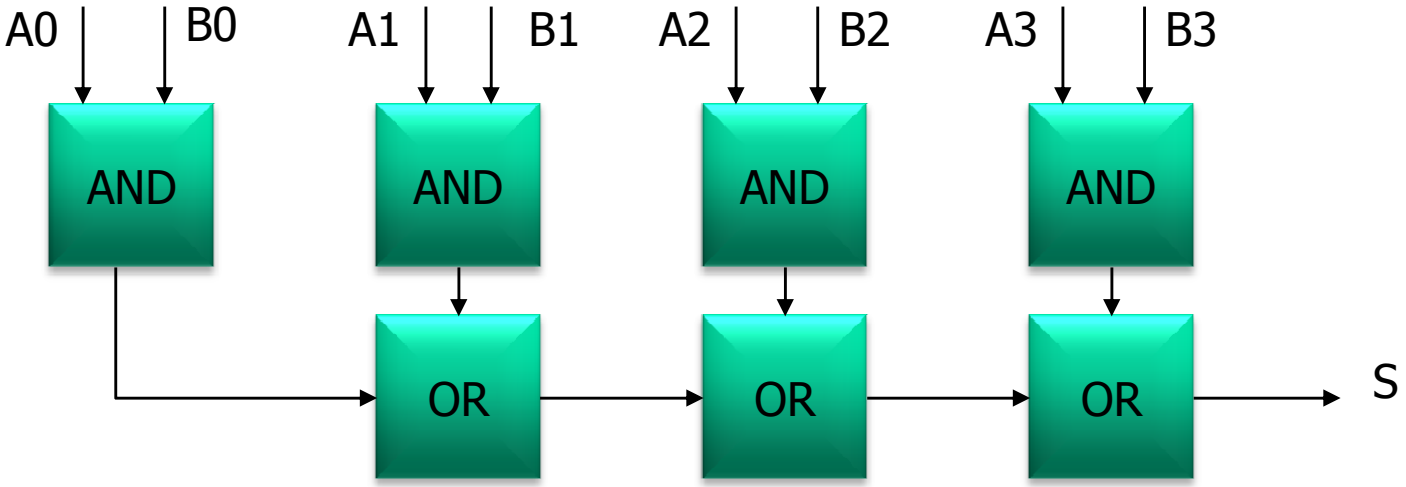
$T_R = 3 T_{RCE} + T_{Com}$.

3 cells.

Synthèse : Nombre de cellule type LUT

Principe :

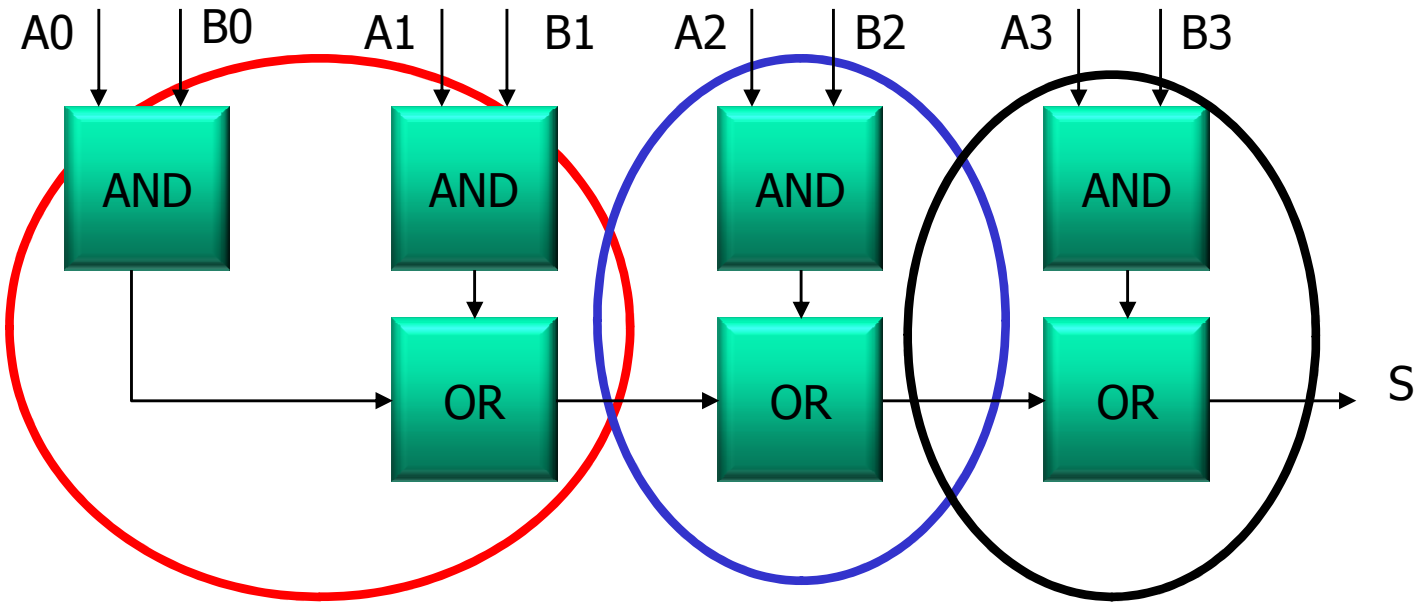
Représentation graphique AND/OR de la fonction



Synthèse : Nombre de cellules type LUT

Regrouper, les portes ET de façon à obtenir K entrées.

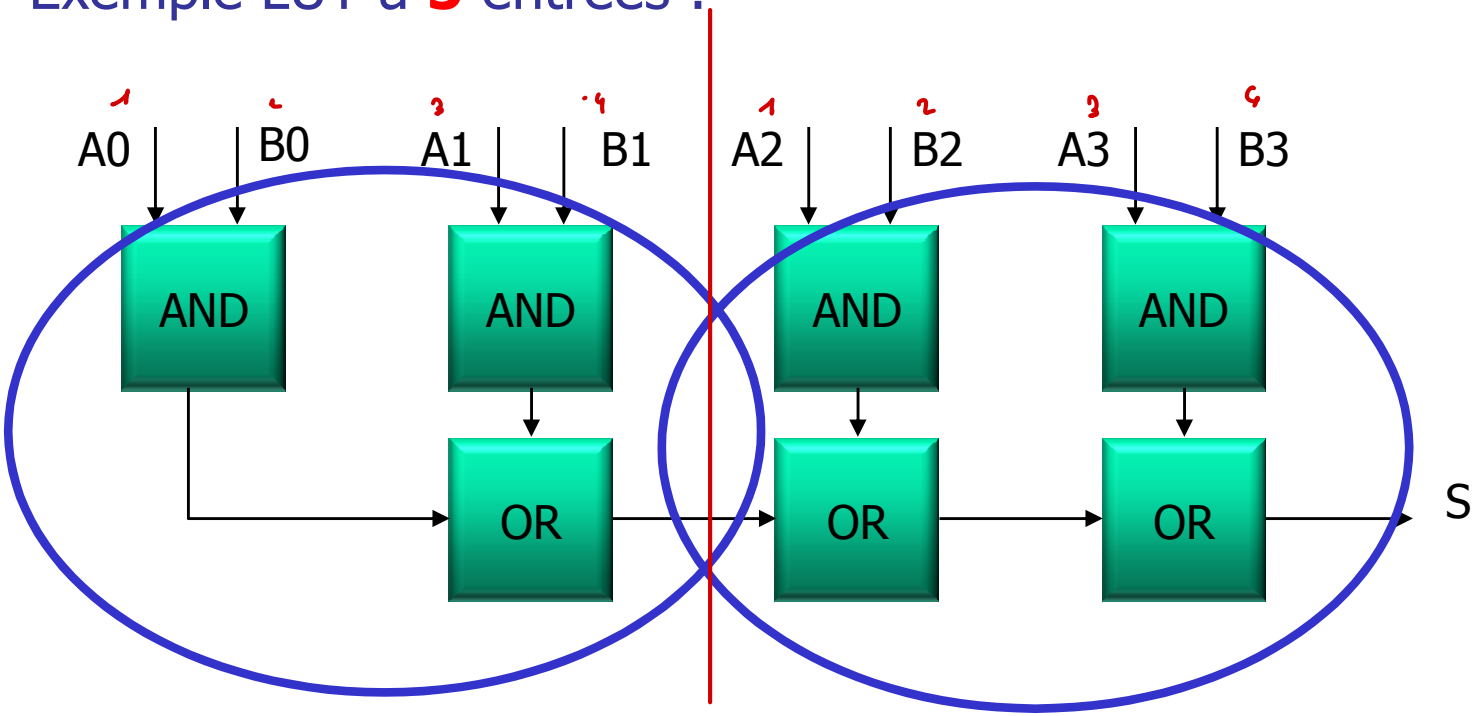
Exemple LUT à **4** entrées :



=> 3 cellules sont nécessaires

Synthèse : Nombre de cellules type LUT

Exemple LUT à **5** entrées :



=> 2 cellules sont nécessaires

Synthèse : Nombre de cellule type LUT

LUT à K entrées (K-LUT)

Représentation graphique AND/OR de la fonction puis :

- *Décomposer les ET comportant plus de K entrées.*
- *Classer de façon décroissante en fonction du nombres d'entrée les portes ET.*
- *Essayer de grouper 2 à 2 les portes ET (groupement ET/OU).*
- *Décomposer la porte OU si le nombre de ces entrées est supérieur à K.*

- Exemple :
  - 2, 3, 4, 3, 4 ; 5-LUT
  - 4, 3, 2, 3 ; 4-LUT, 6-LUT

$F_1 = \underbrace{\bar{A}\bar{B}\bar{C} + ABC\bar{E}}_{F_1'} + ACF + DE\bar{F}$ Lut : 4 entrées

$F_1' = \bar{A}\bar{B}\bar{C} + ABC\bar{E}$.

$F_{11}' = F_1' + ACF$

$F_1' = F_{11}' + DE\bar{F}$

} 3 cells

$F_1 = \underbrace{\bar{A}\bar{B}\bar{C} + ABC\bar{E} + ACF}_{F_1'} + DE\bar{F}$ Lut : 5 entrées

$F_1' = \bar{A}\bar{B}\bar{C} + ABC\bar{E} + ACF$

$F_1 = F_1' + DE\bar{F}$

} 2 cells

$TR = 2 \times T_{acc} + T_{com}$.

Synthèse avec LUT : Entrées dépendantes

Entrées dépendantes :

Décomposition en arbre binaire par rapport à certaines variables d'entrées

F_1 = !A.!B.!C + A.B.C.E + A.C.F + D.E.!F

LUT à 4 entrées : ? LUT à 5 entrées : ?

Nombre de cellules ?

Idem pour : F_2 = !A.!B.!C.D.E + A.B.C.D.E + A.C.D.F + B.C.D.E.!F

Pour s'entrainer : registre avec chargement //, décalage à droite, à gauche, mémorisation, clear, preset synchrones.

Idem pour : F_2 = !A.!B.!C.D.E + A.B.C.D.E + A.C.D.F + B.C.D.E.!F

$\bar{A}\bar{B}\bar{C}DE + ABCDE$

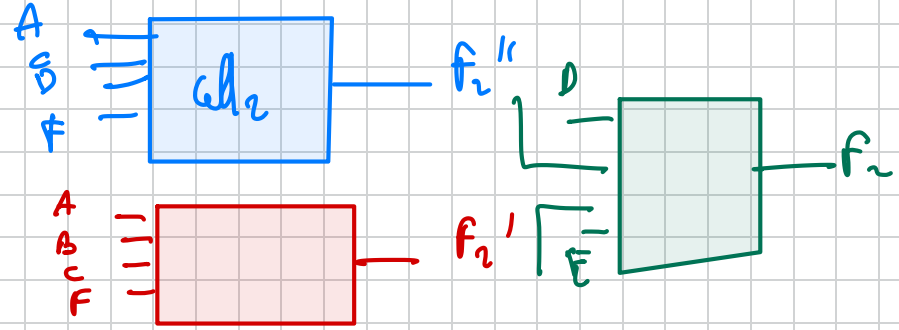
$= DE(\bar{A}\bar{B}\bar{C} + ABC + BC\bar{F}) + ACDF$

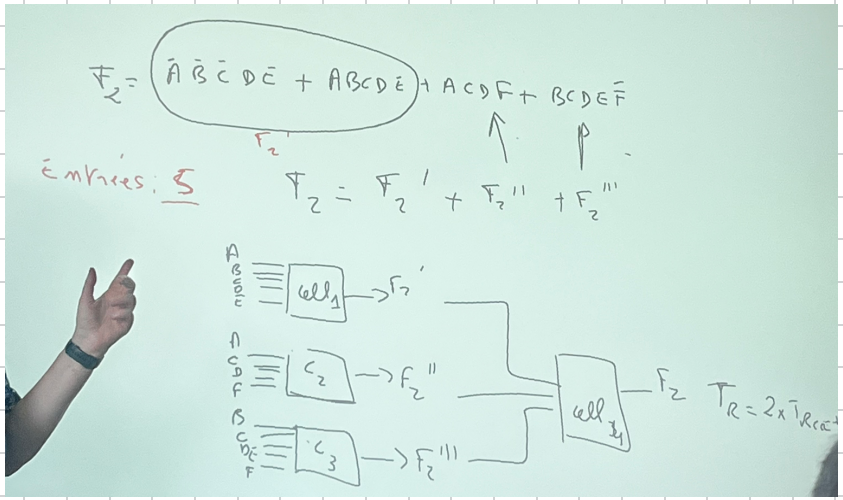
$= DE F_2' + ACDF$

$F_2 = DE F_2' + F_2''$

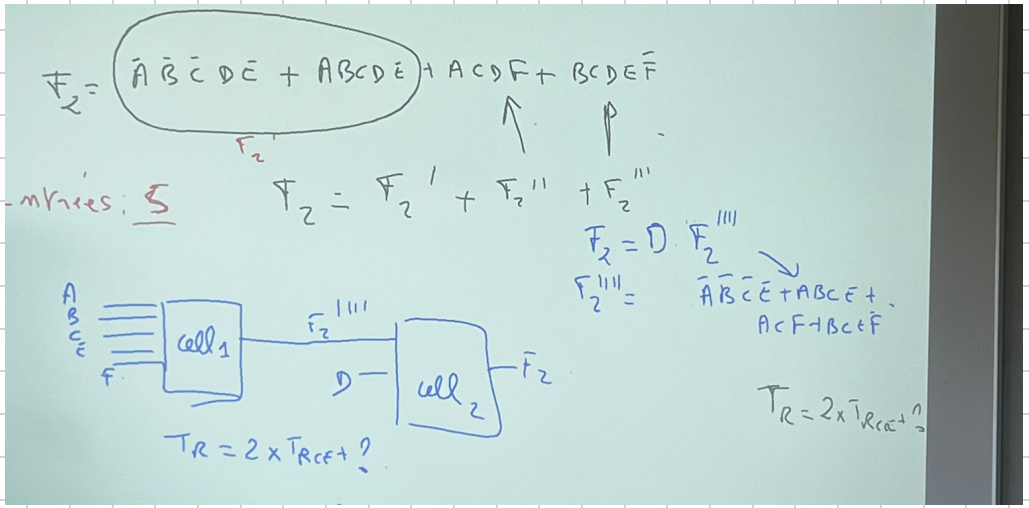
$T_R = 2\,T_{RCE} + \ldots$

Lourd à calculer.
On est bon à 2 cellules.





1 cas



2 ou + cas

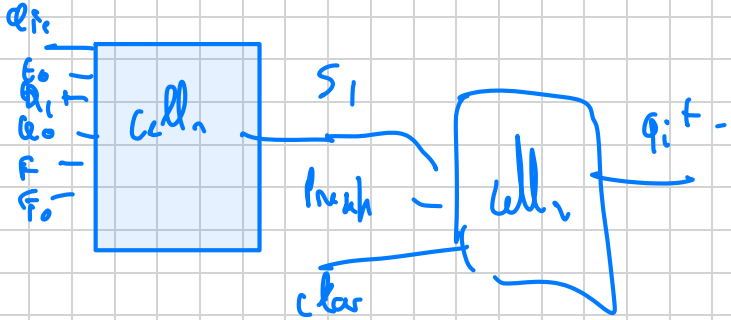
$$Q_i^+ = clear(\bar{F}_1 \bar{F}_0 Q_i + \bar{F}_1 F_0 \epsilon_i + F_1 \bar{F}_0 Q_{i+1} + F_1 F_0 Q_{i-1} + Reset)$$

8 entrées pour Q_i^+.

LUT : 6 entrées.

$Q_i^+ = clear(S_1 + Reset)$

$t_a = 2 t_{LUT} + t_{carry}$.



$$Q_i^+ = clear(\underbrace{\bar{F}_1 \bar{F}_0 Q_i + \bar{F}_1 F_0 \epsilon_i}_{S_2} + \underbrace{F_1 \bar{F}_0 Q_{i+1} + F_1 F_0 Q_{i-1}}_{S_3} + Reset)$$

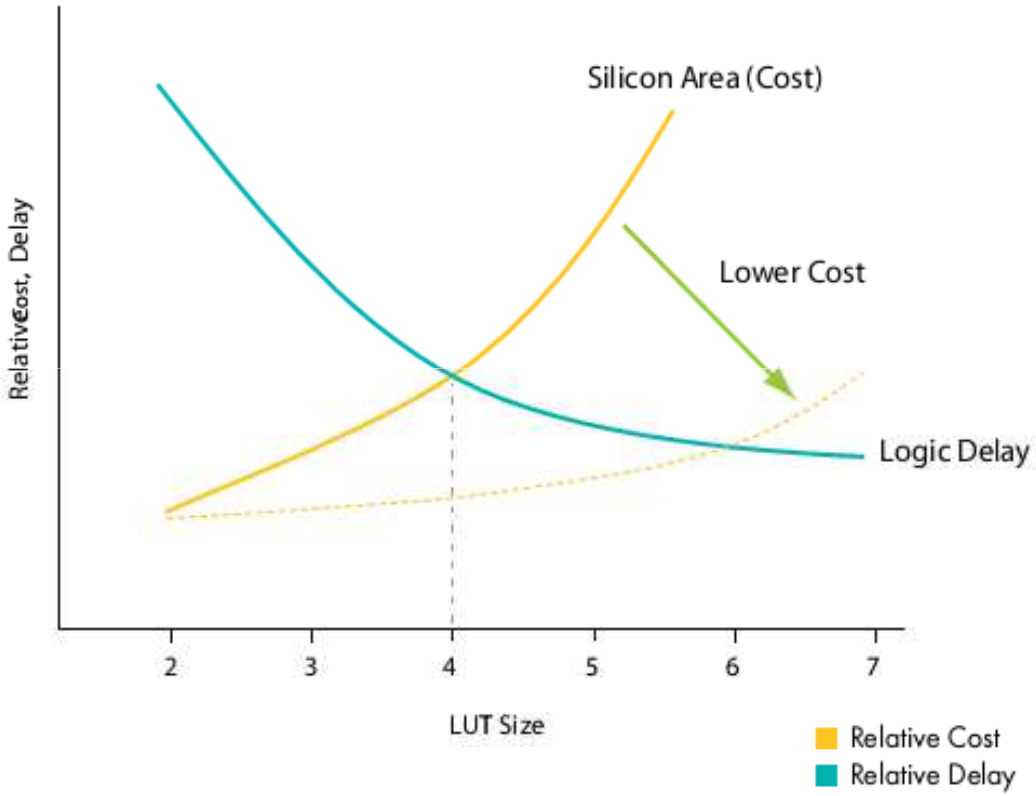
8 entrées pour Q_i^+.

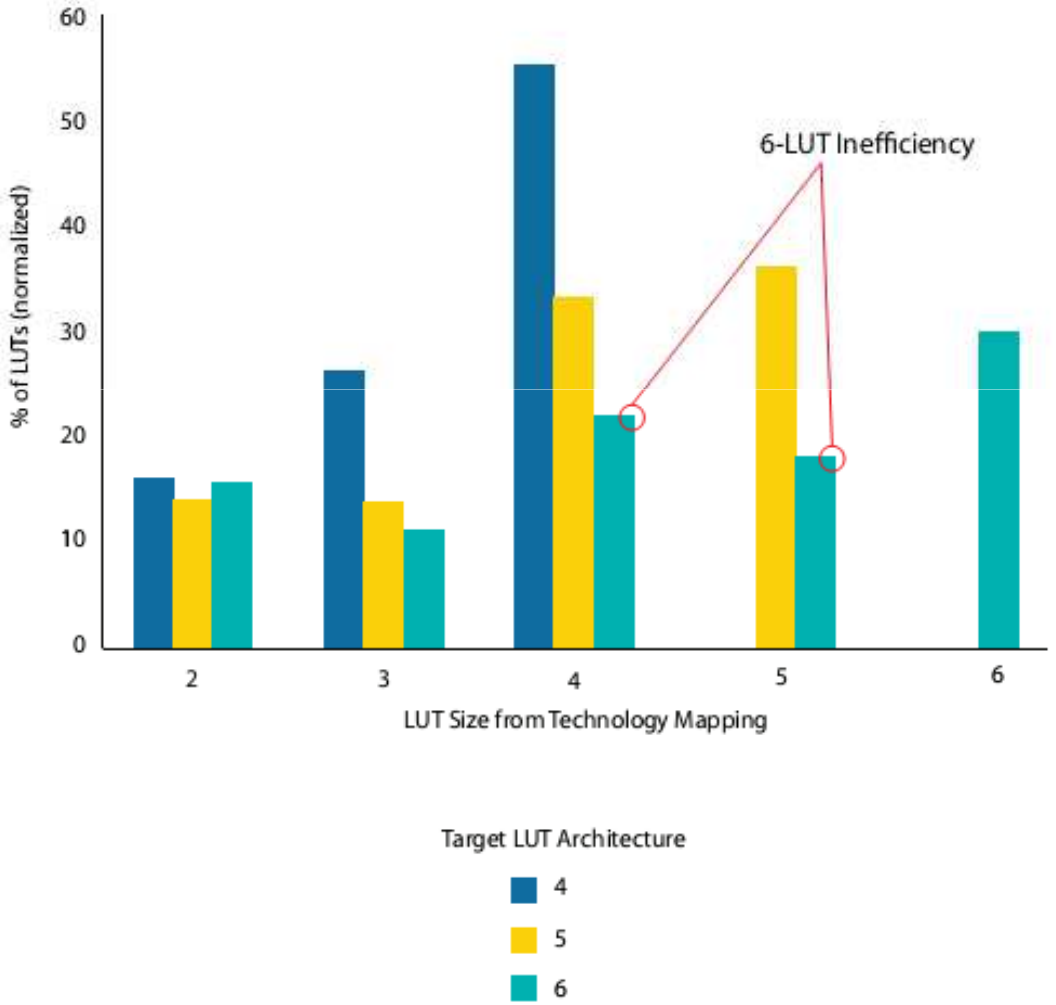
LUT : 4 entrées

$t_a = 2 \times t_{LUT} + t_{comp}$.

$Q_i^+ = clear(S_2 + S_3 + Reset)$

Influence du nombre d'entrées des LUT sur la synthèse



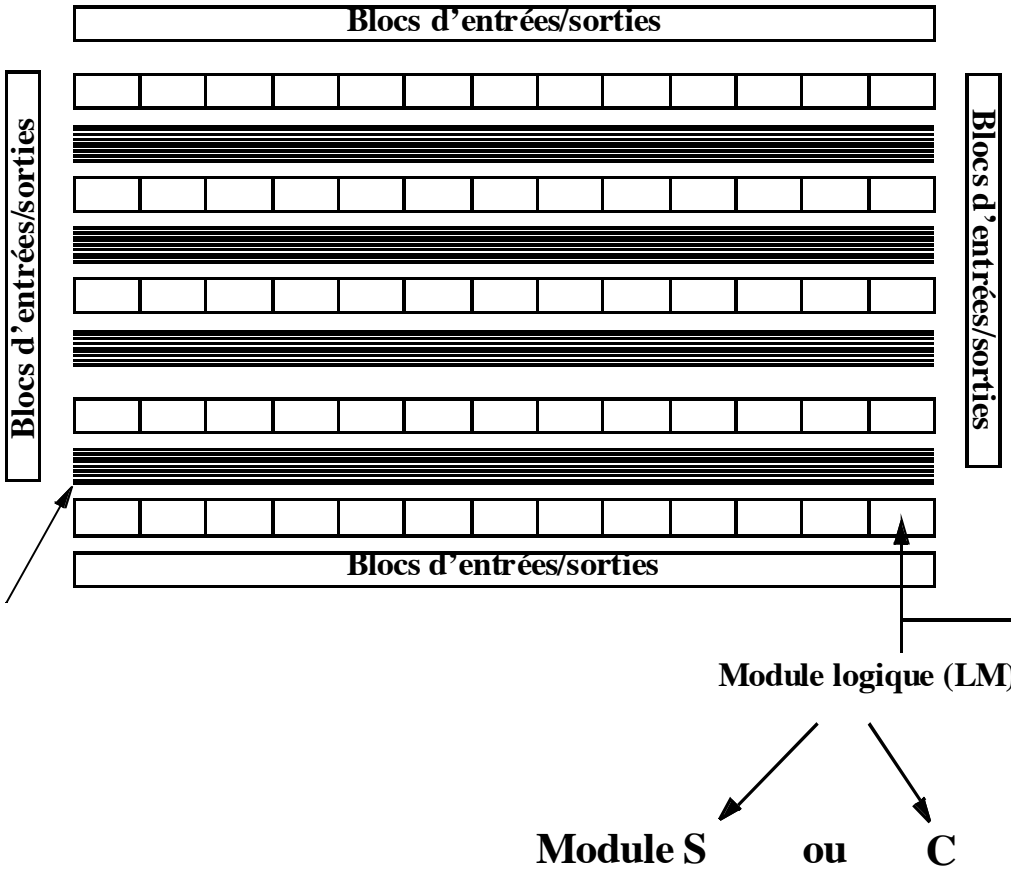


Source : Altera/Intel

Les FPGAs

- **Cellules avec LUTs:**
  - **XILINX : famille XC 2000, XC3000, XC4000**
  - **ALTERA : Famille FLex,**
- **Cellules avec MPX**
  - **ACTEL : famille Act1 et ACT2**

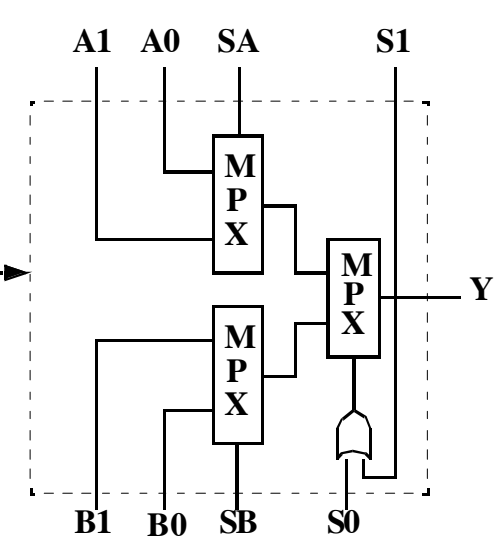
Architecture ACTEL (1) : famille Act1



Technologie Anti-fusible.

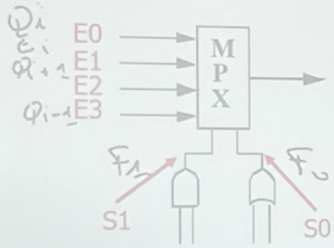
Interconnection :

- **canaux de routage,**
- **13 lignes verticales au dessus de chaque module.**



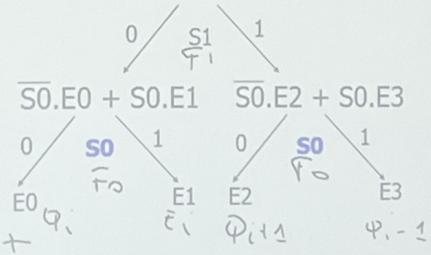
Passage par une Bascule D si module S

Famille avec MPX : Méthode de Synthèse



$\Rightarrow Y = \overline{S1}.\overline{S0}.E0 + \overline{S1}.S0.E1 + S1.\overline{S0}.E2 + S1.S0.E3$

⇒ **Décomposition sous forme d'arbre binaire**



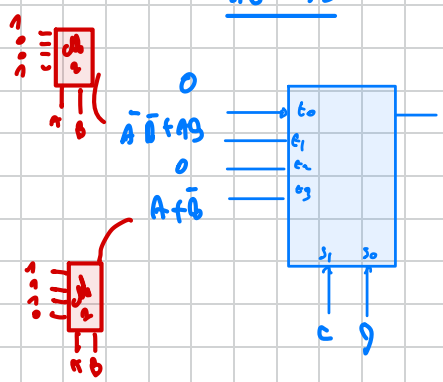
$Q_{i+1} = \overline{F_1}F_0 Q_i + \overline{F_1}F_0 E_i + F_1\overline{F_0} Q_{i+1} + F_1 F_0 Q_{i-1}$ ⇒ Identification par rapport aux fonctions à réaliser

$Y = \bar{A}\bar{B}\bar{C}D + A\bar{B}\bar{C}D + \bar{A}CD + \bar{B}CD$

arbre binaire à 2 niv.

$\bar{A}\bar{B}D + ABD$ | $AD + \bar{B}D$

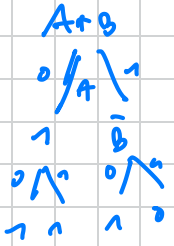
$\bar{A}\bar{B} + AB$ | $A\bar{B}$



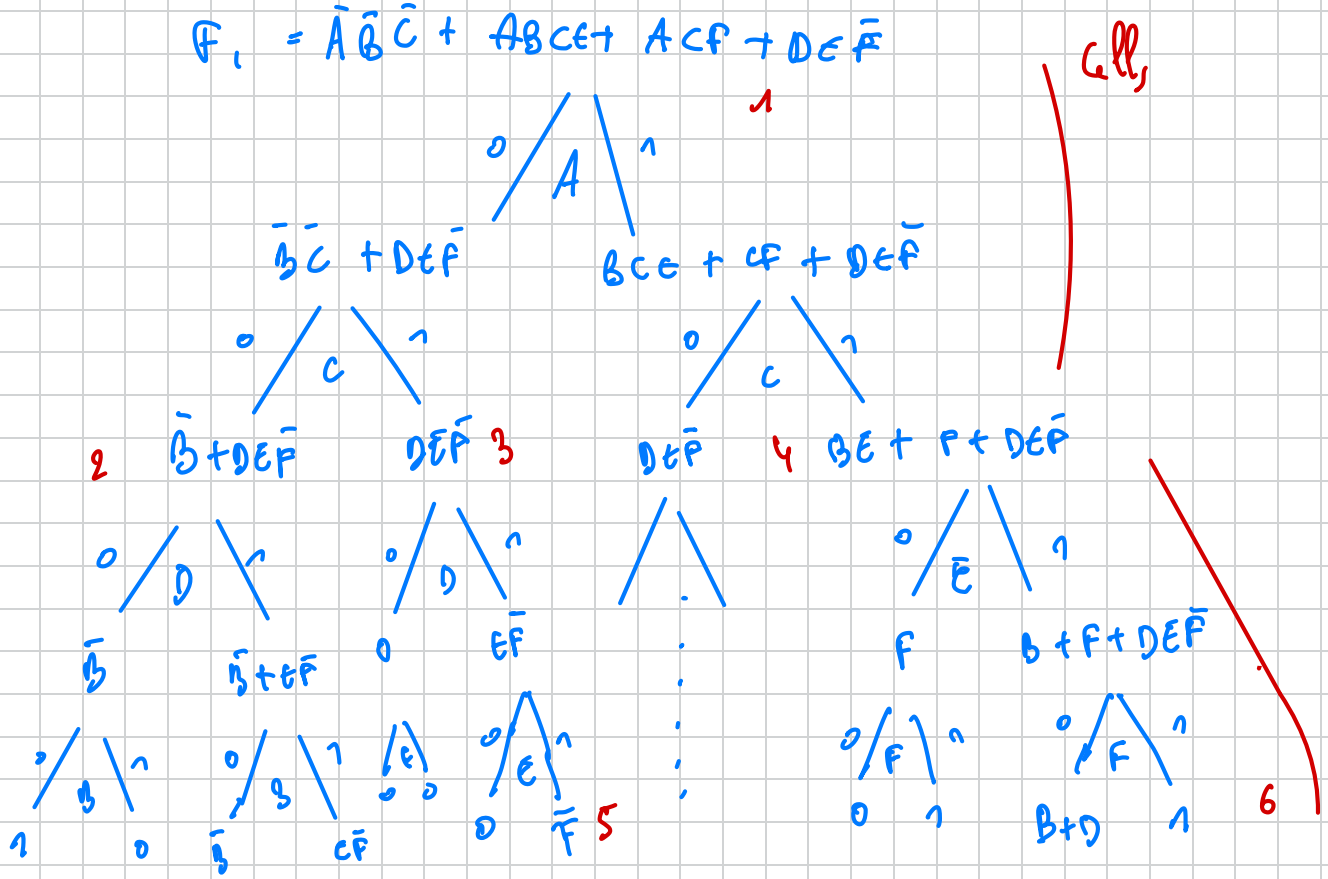
Cell : 1 $A\bar{B} + \bar{A}\bar{B}$



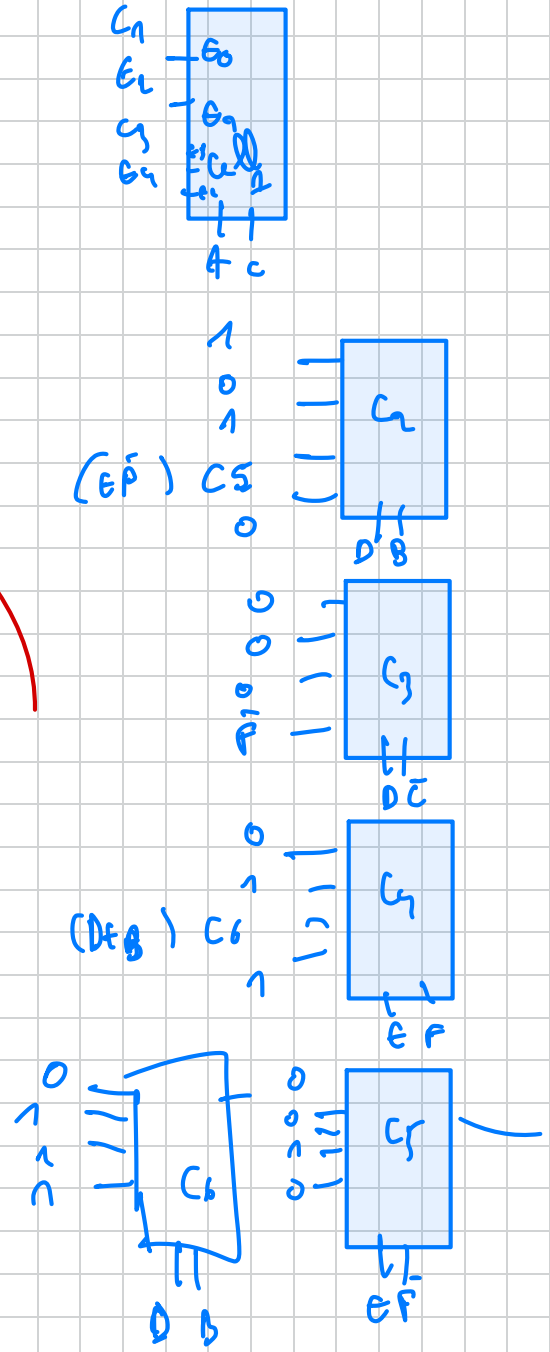
Cell$_2$:



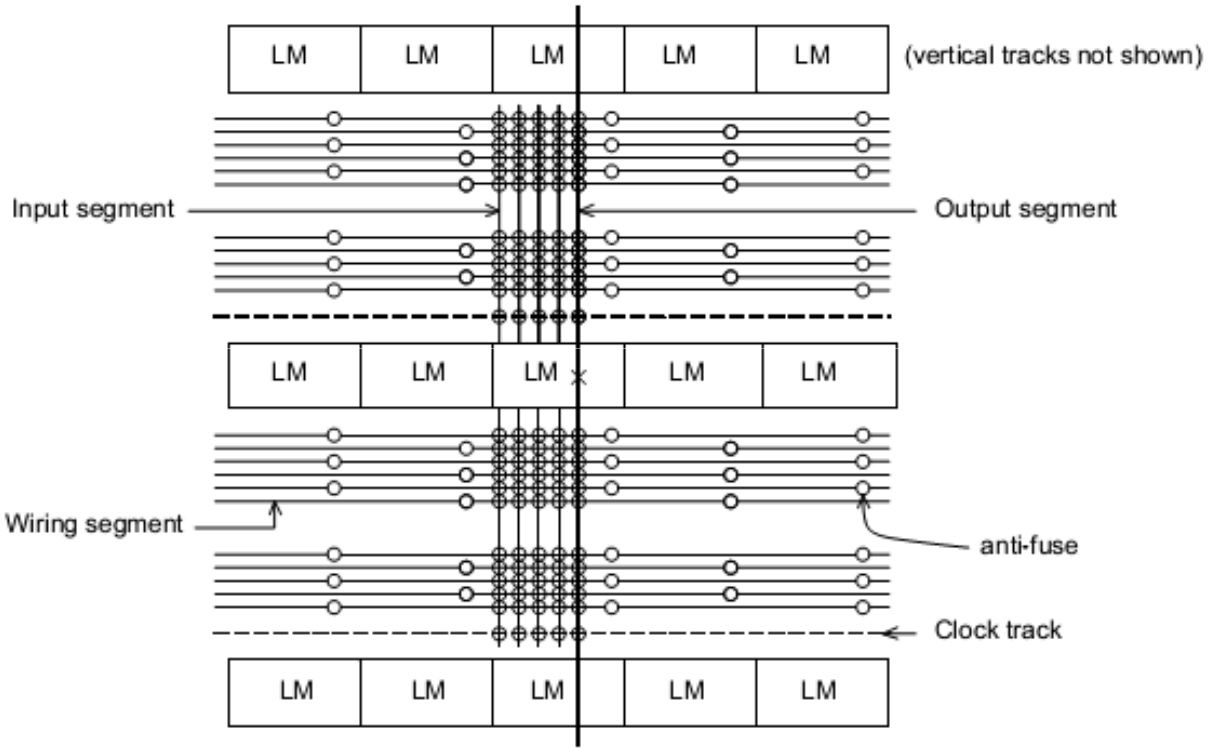
$F_1 = \bar{A}\bar{B}\bar{C} + ABC\bar{E} + ACF + DE\bar{F}$



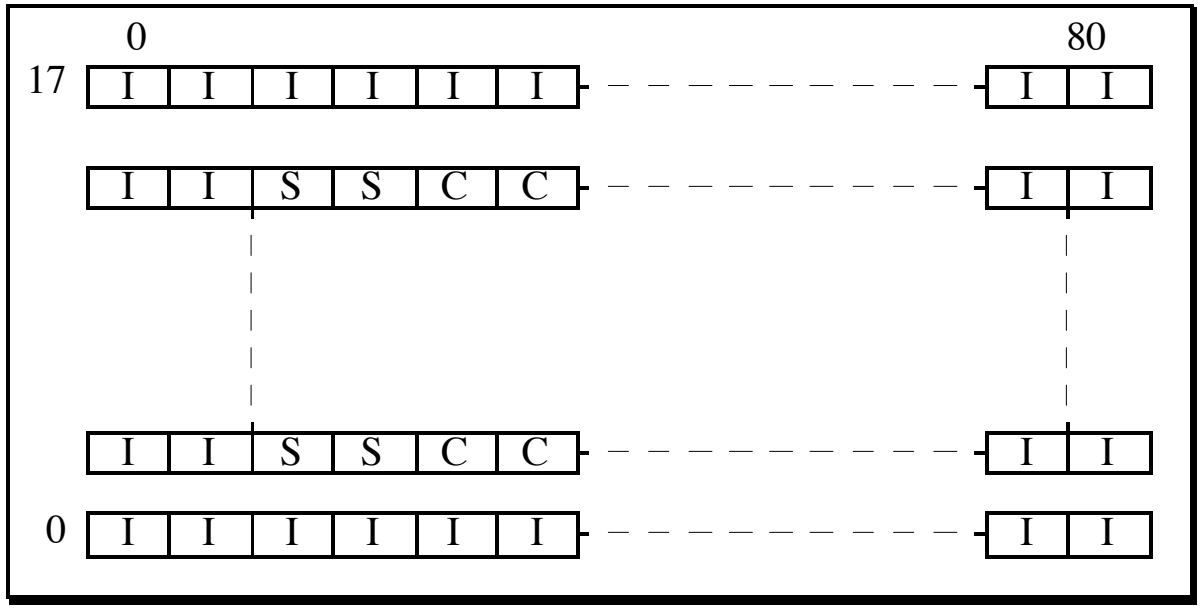
$f_Q = 3$ to 2 MPX $+ ?$

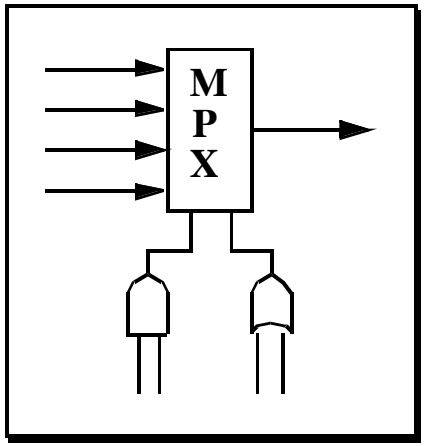


Famille Actel interconnexion

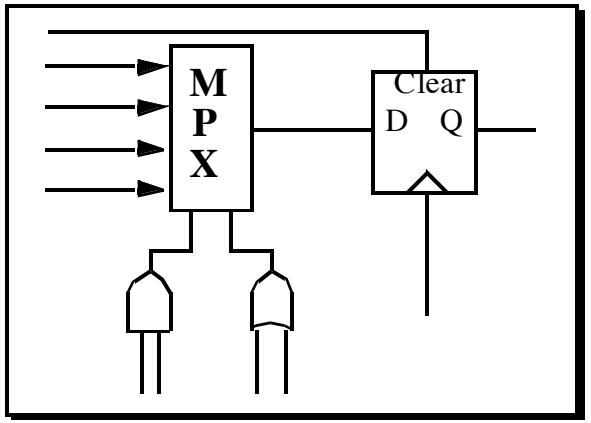


Architecture ACTEL (2) : famille Act2, Act3



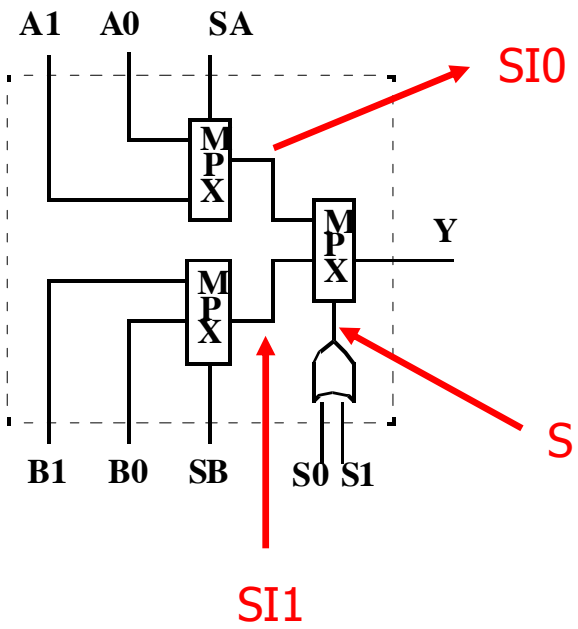


Module C



Module S

Famille avec MPX : Synthèse – Act1



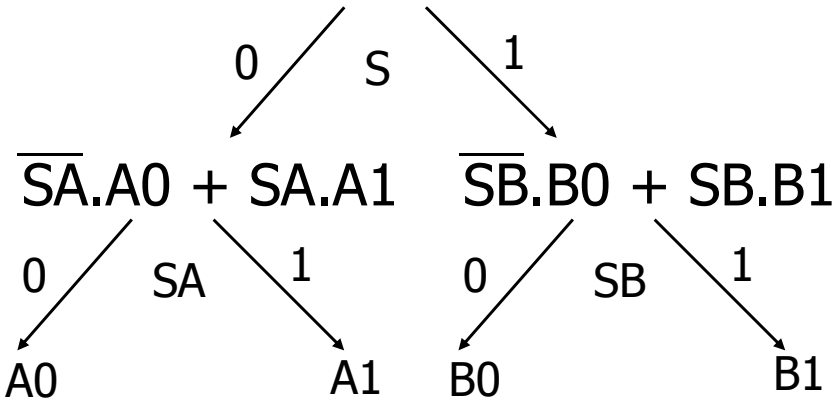
$Y = \overline{S}.SI0 + S.SI1$

$SI0 = \overline{SA}.A0 + SA.A1$

$SI1 = \overline{SB}.B0 + SB.B1$

$\Rightarrow Y = \overline{S}.(\overline{SA}.A0 + SA.A1) + S.(\overline{SB}.B0 + SB.B1)$

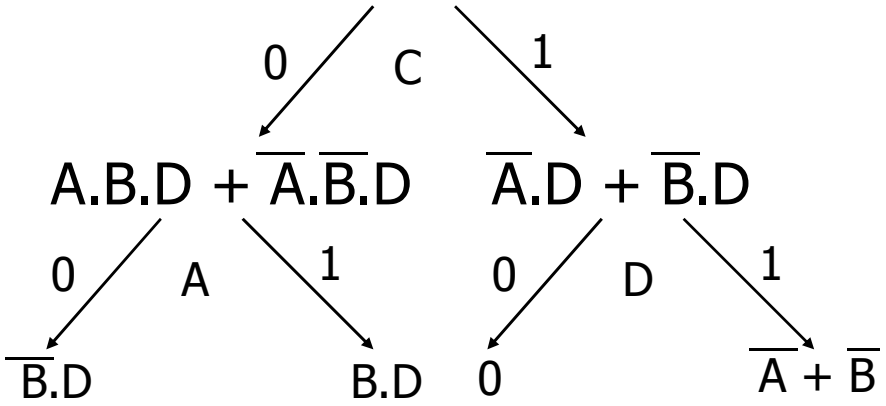
$\Rightarrow$ **Décomposition sous forme d'arbre binaire**



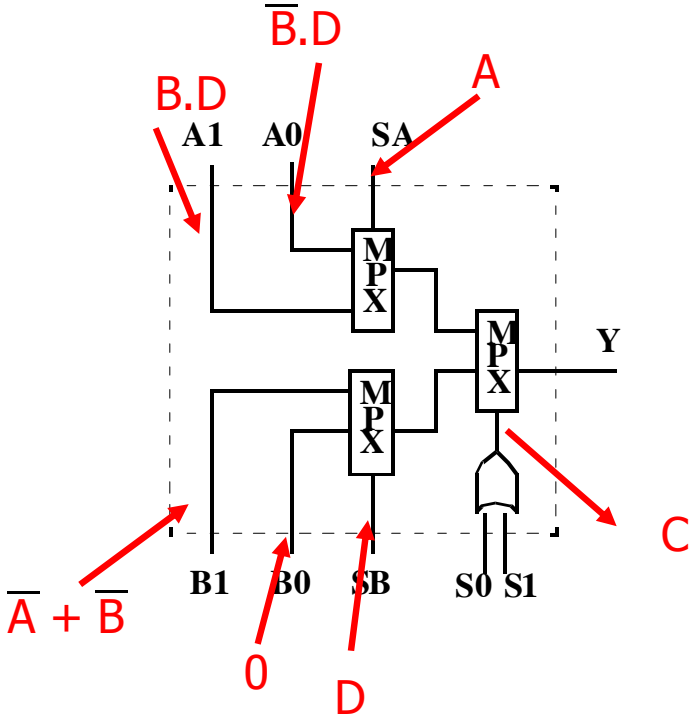
=> Identification par rapport aux fonctions à réaliser

Famille avec MPX : Synthèse – Act1

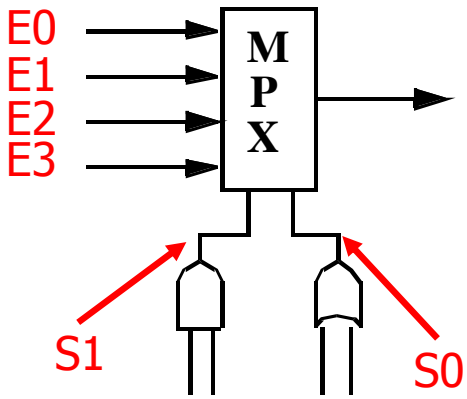
$$Y = \overline{A}.\overline{B}.\overline{C}.D + A.B.\overline{C}.D + \overline{A}.C.D + \overline{B}.C.D$$



=> 4 cellules pour réaliser cette fonction

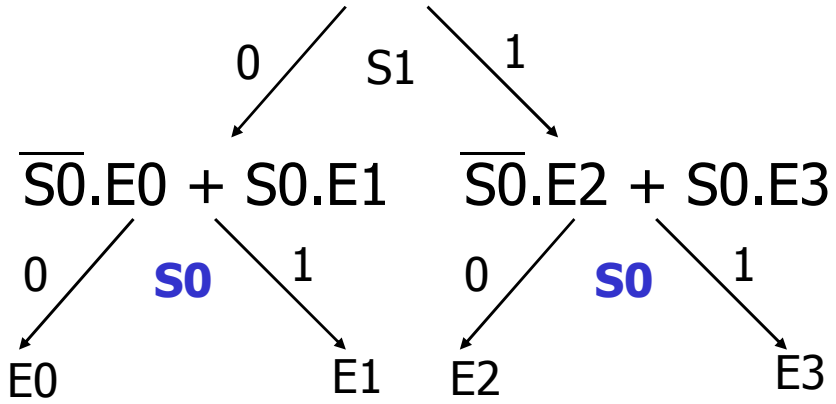


Famille avec MPX : Synthèse – Act2



$\Rightarrow Y = \overline{S1}.\overline{S0}.E0 + \overline{S1}.S0.E1 + S1.\overline{S0}.E2 + S1.S0.E3$

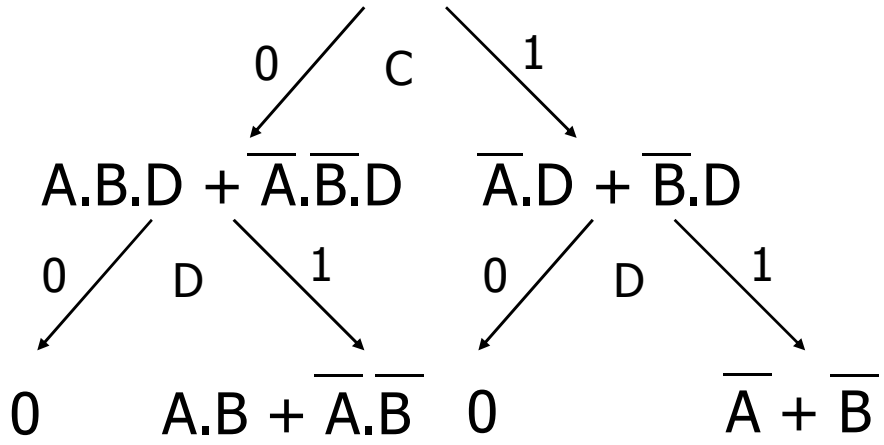
$\Rightarrow$ **Décomposition sous forme d'arbre binaire**



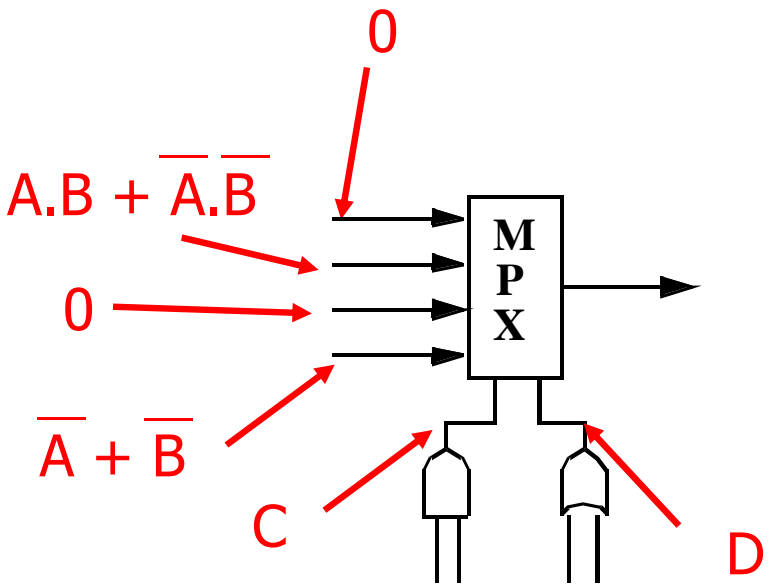
=> Identification par rapport aux fonctions à réaliser

Famille avec MPX : Synthèse – Act2

$$Y = \overline{A}.\overline{B}.\overline{C}.D + A.B.\overline{C}.D + \overline{A}.C.D + \overline{B}.C.D$$

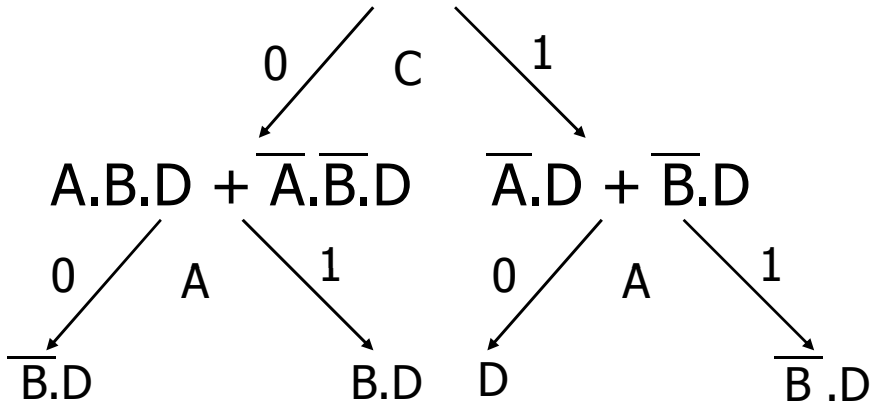


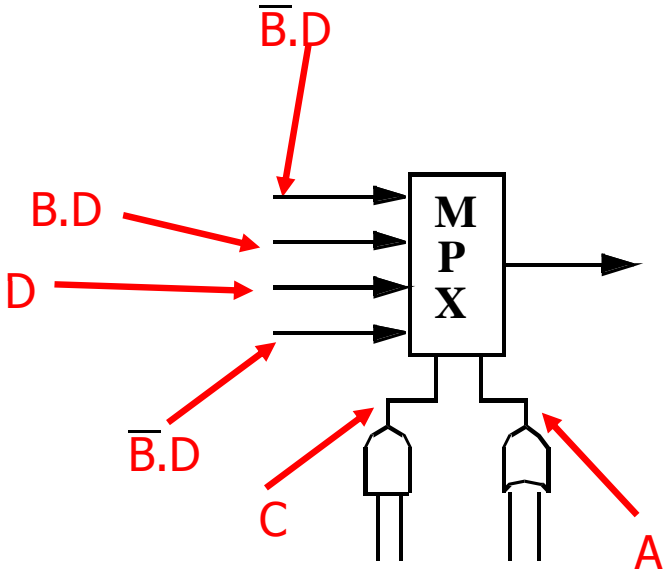
=> 3 cellules pour réaliser cette fonction



Famille avec MPX : Synthèse – Act2

$$Y = \overline{A}.\overline{B}.\overline{C}.D + A.B.\overline{C}.D + \overline{A}.C.D + \overline{B}.C.D$$





=> 3 cellules pour réaliser cette fonction

Exercice:
Effectuer la synthèse pour les fonctions suivantes
- F_1 = !A.!B.!C + A.B.C.E + A.C.F + D.E.!F
- F_2 = !A.!B.!C.D.E + A.B.C.D.E + A.C.D.F + B.C.D.E.!F
- registre avec chargement //, décalage à droite, à gauche, mémorisation, avec clear et preset synchrones.

Autres cellules

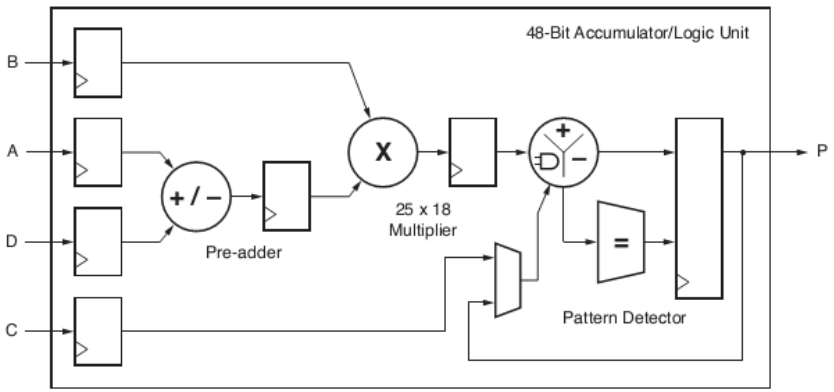
- Blocs RAM
- Blocs DSP : Traitement Image, Traitement audio => multiplication
- Exemple famille Intel-Altera/Stratix :

  Blocs de mutiplieurs :9x9, 18x18, 27x27, 32x32

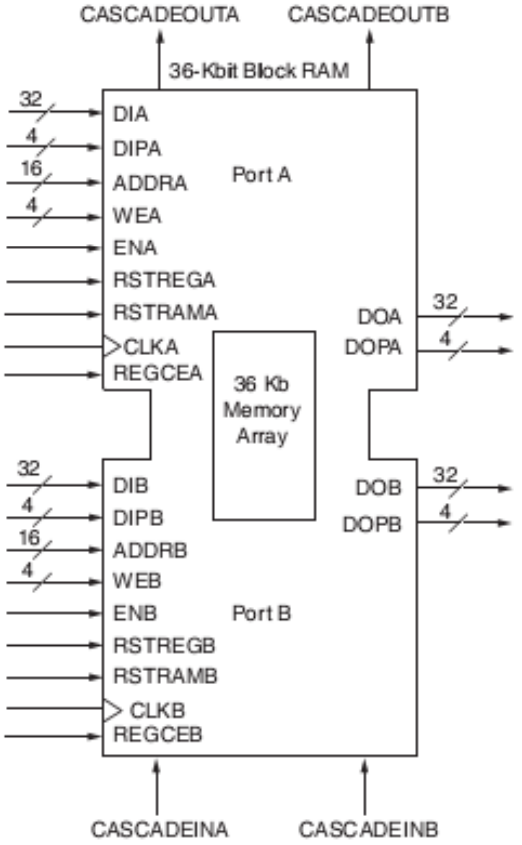
  Stratix V : 12, 8, 4, 2,

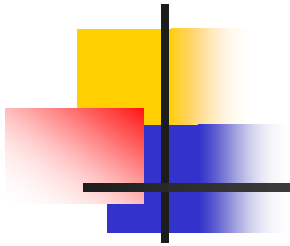
⇒ Accélération : transformée de fourrier, coder/encoder vidéo, …

⇒ PLL, PCI, Ethernet, …



mac : S = S + A×B





Placement et routage

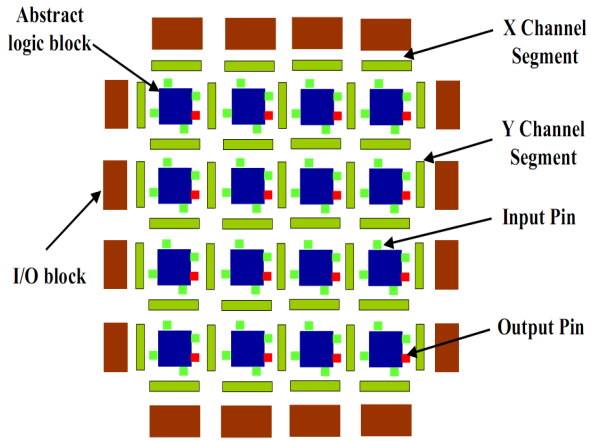
Placement des fonctions logiques dans un FPGA

Points d'entrée :

- Liste de blocs logiques
- Nombre d'entrée/sortie
- Communication entre les blocs

But du placement :

- déterminer la position en X, Y de chaque bloc
- Interconnexion : type, longueur
- Evaluation du temps additionnel liée à la communication entre les cellules



Placement des fonctions dans un FPGA

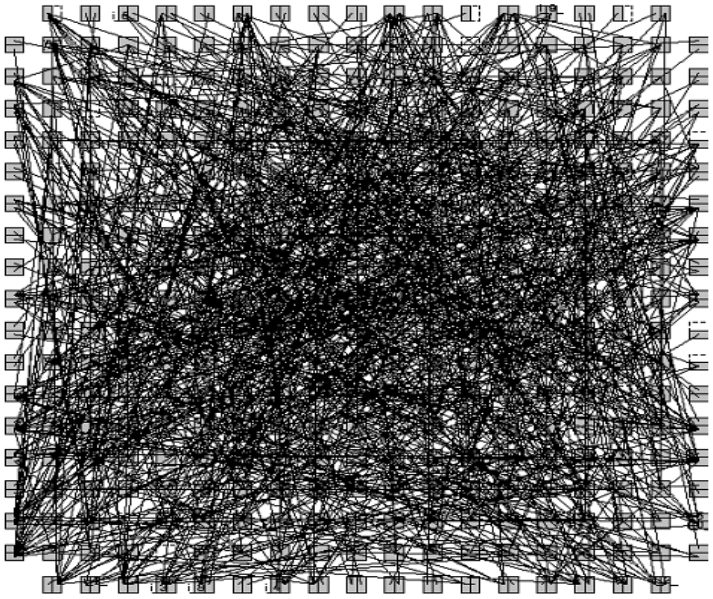
- Priorité : temps de réponse

- Calcul d'une fonction de coût par paire de cellules. On place en priorité les fonctions qui ont la fonction de coût la plus élevée.

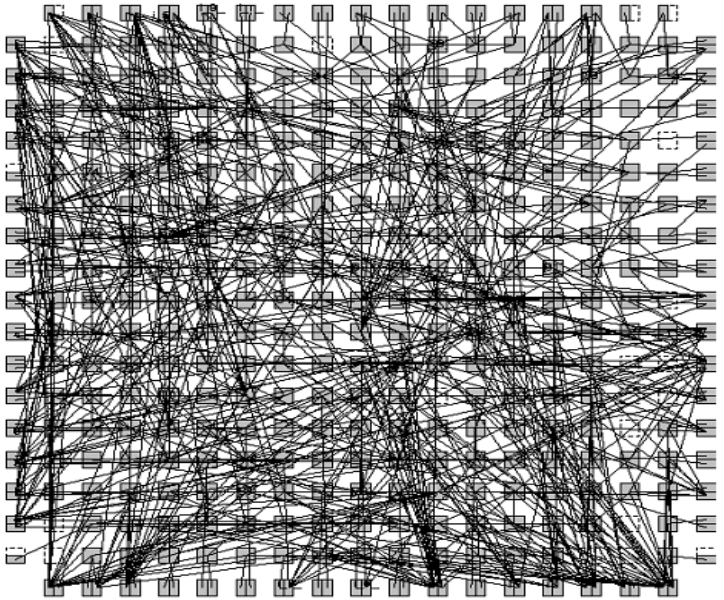
- Par cluster de graphe.

  - Vérification des contraintes temporelles ?

Placement random



Placement final

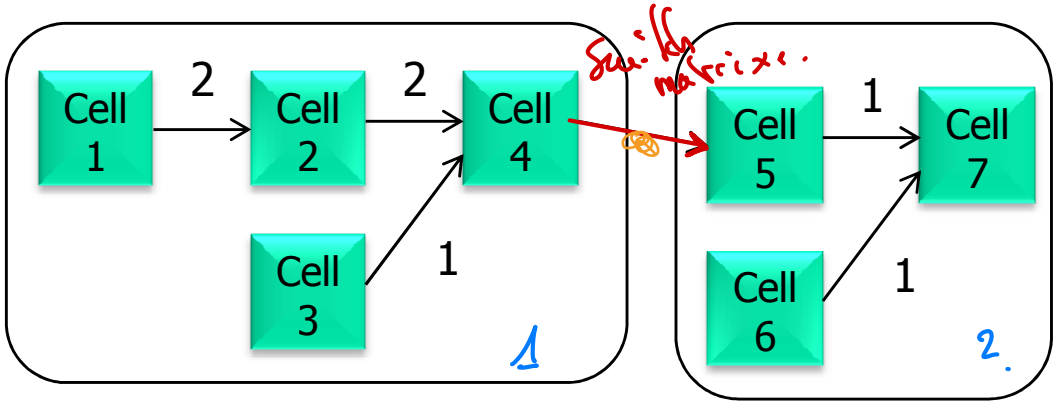


Algorithme de placement

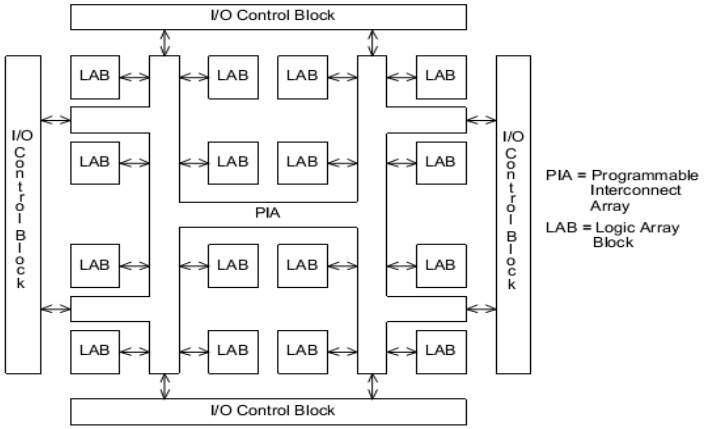
1- Exemple pour CPLD/FPGA-cluster

Principe : Affectation des cellules aux clusters => Partitionnement des graphes des cellules en sous-graphes ayant un nombre minimum de connexions

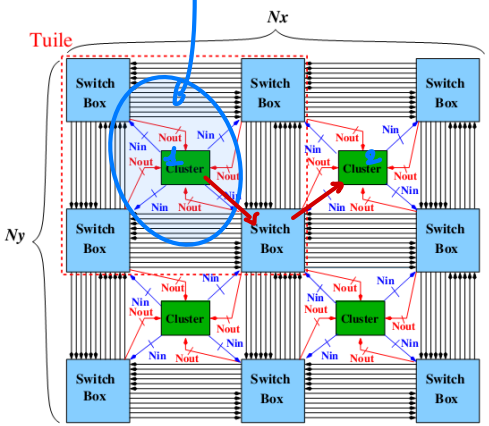
Cluster 1



Cluster 2



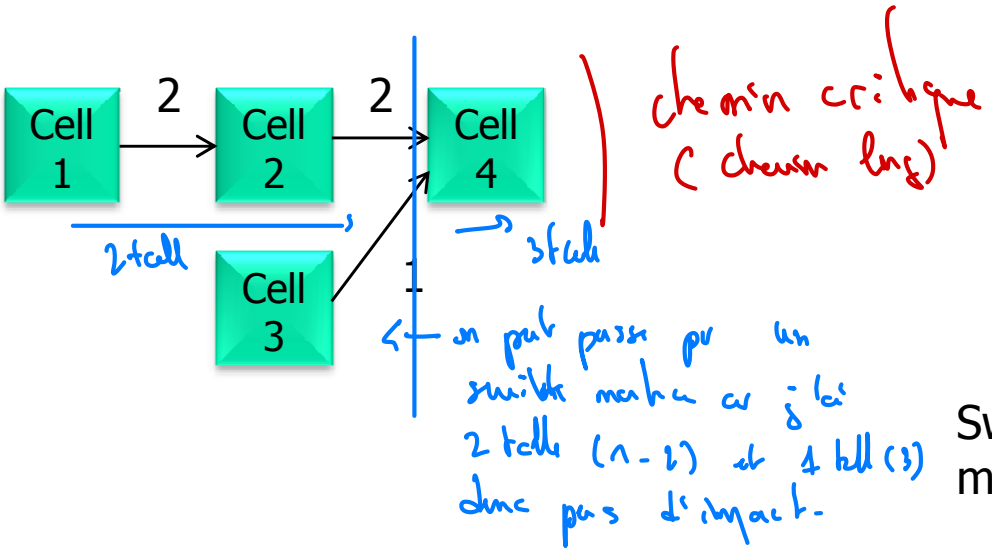
Exemple d'algorithme : min-cut

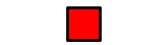


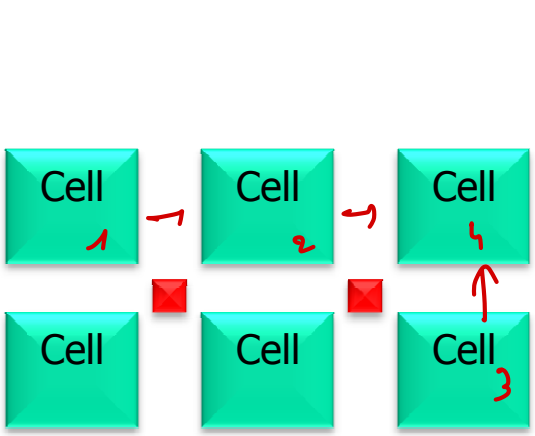
Algorithme de placement

2 - Exemple pour FPGA :

associée aux communications une fonction de coût représentant une priorité de placement notamment de façon à réduire le critical path (chemin critique).

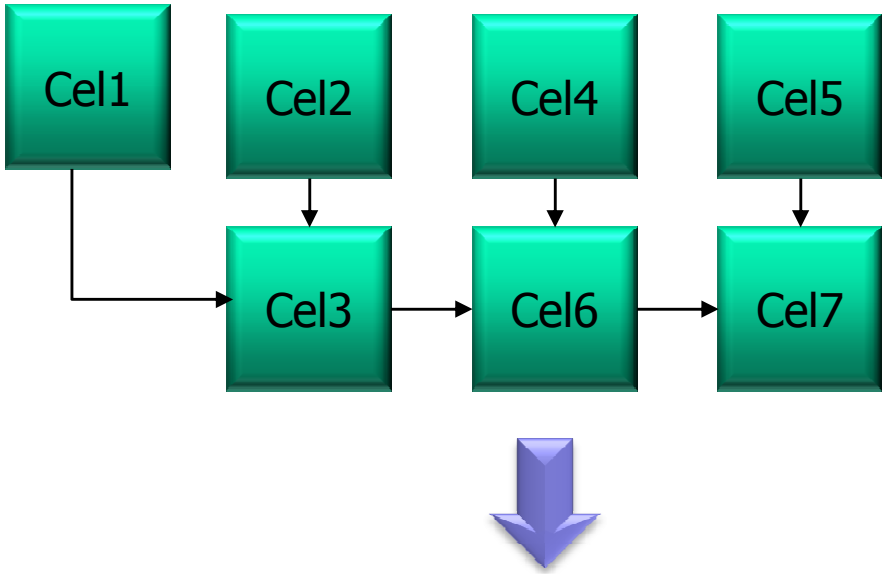






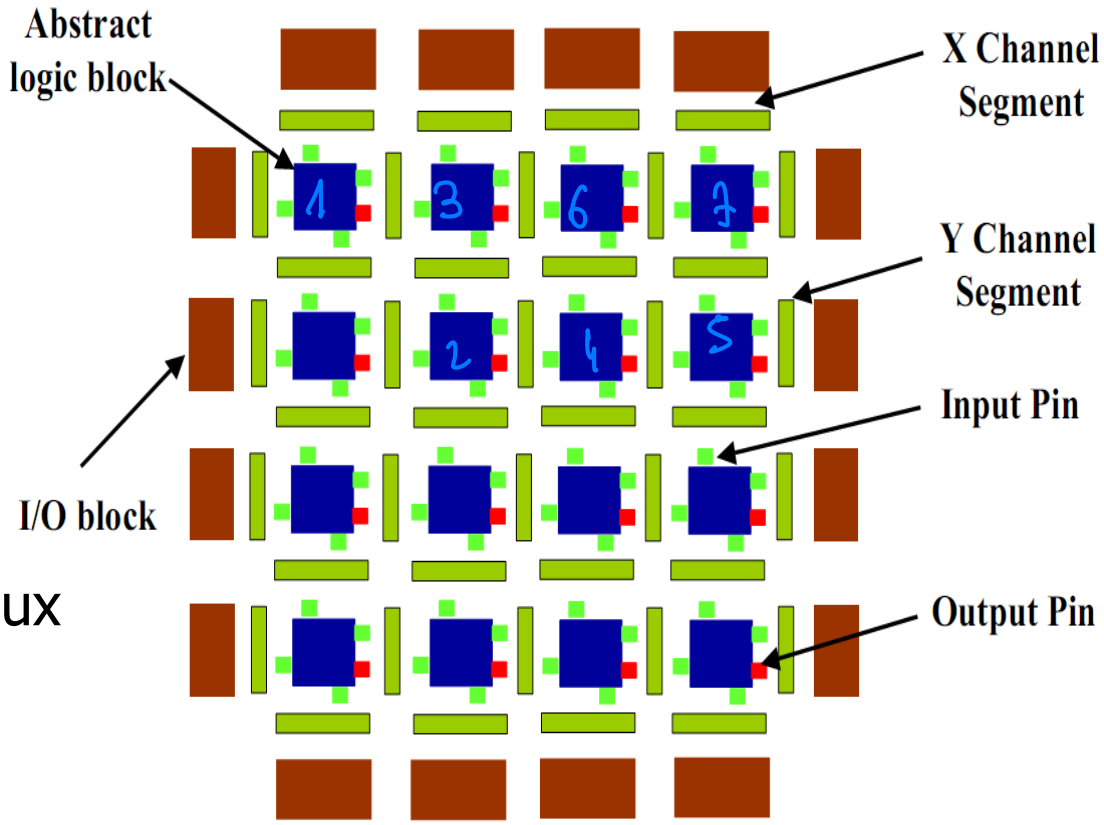
Placement des fonctions logiques dans un FPGA

- Exemple :



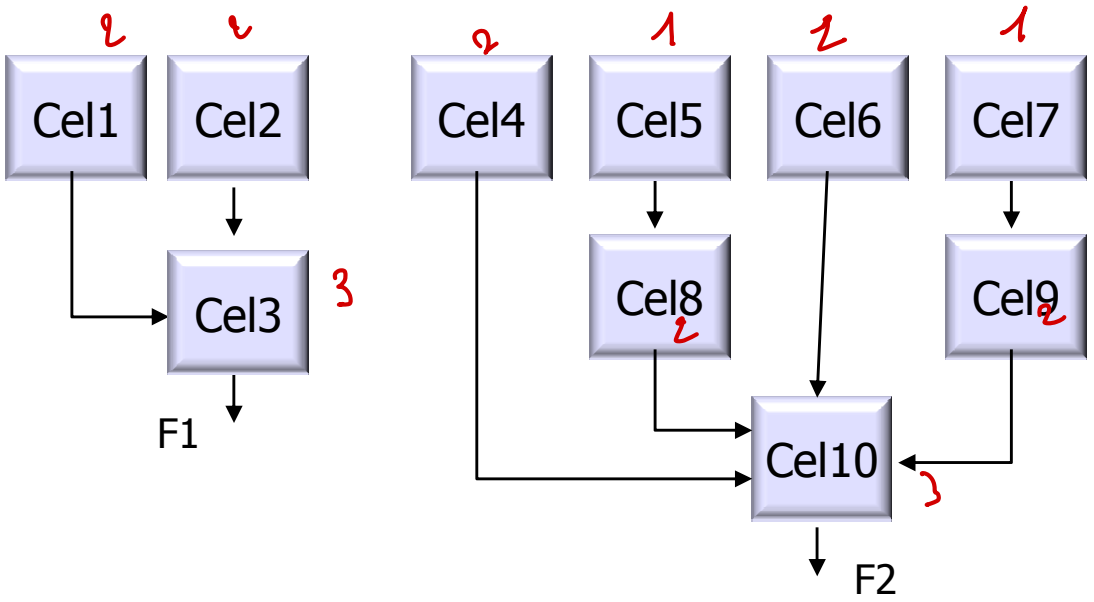
Placement ?

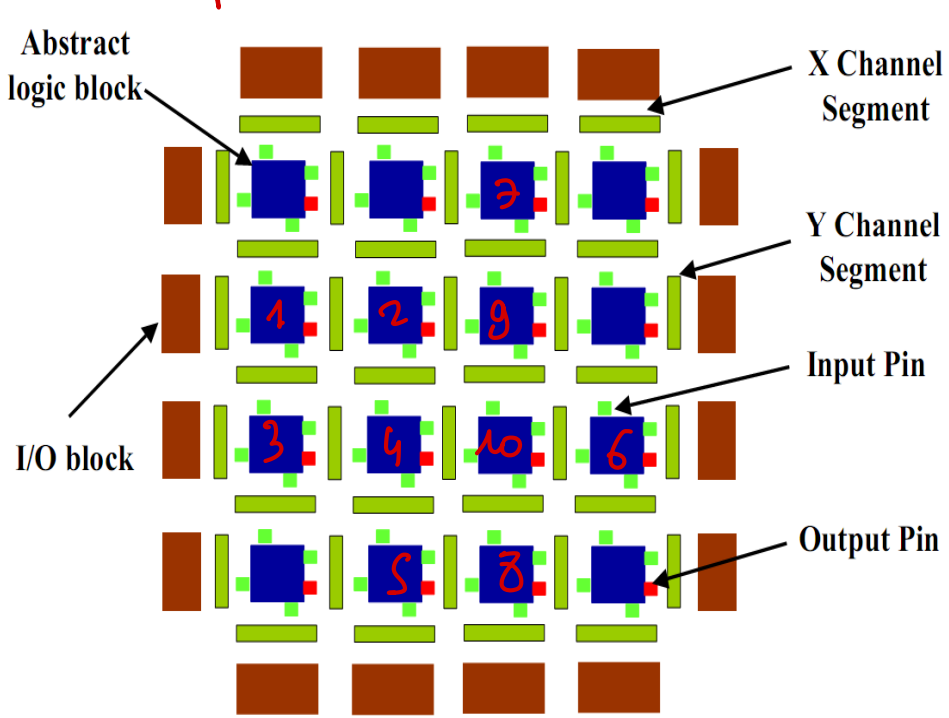
Evaluer le temps de réponse dans les deux cas suivants :
- par ordre de numérotation des cellules
- en considérant le chemin critique ?



Placement des fonctions logiques dans un FPGA

- Exemple : F1, F2, additionneur 8 bits, compteur 8 bits

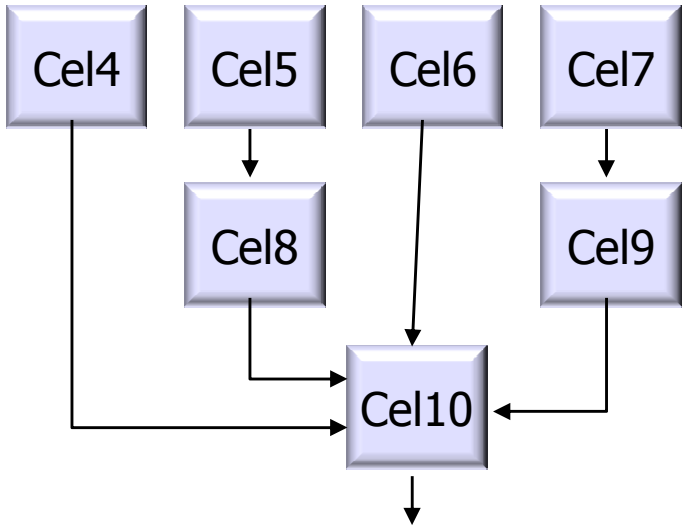


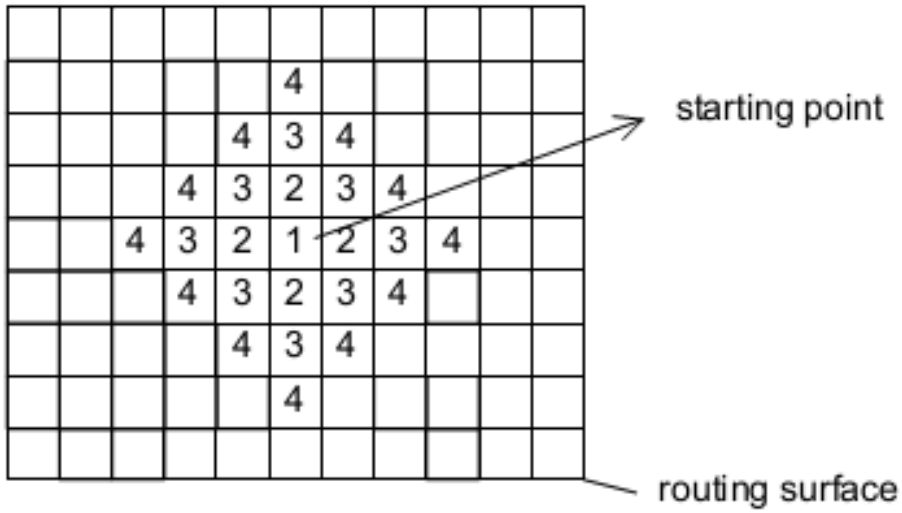


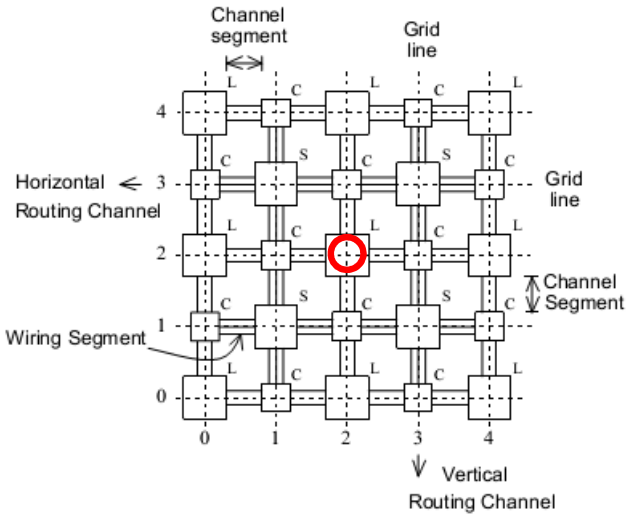
Placement ?

Algorithme de placement/routage

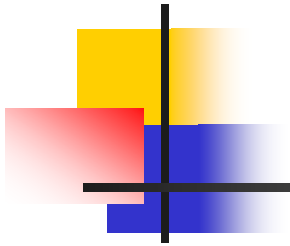
- Maze router (Diamond) :





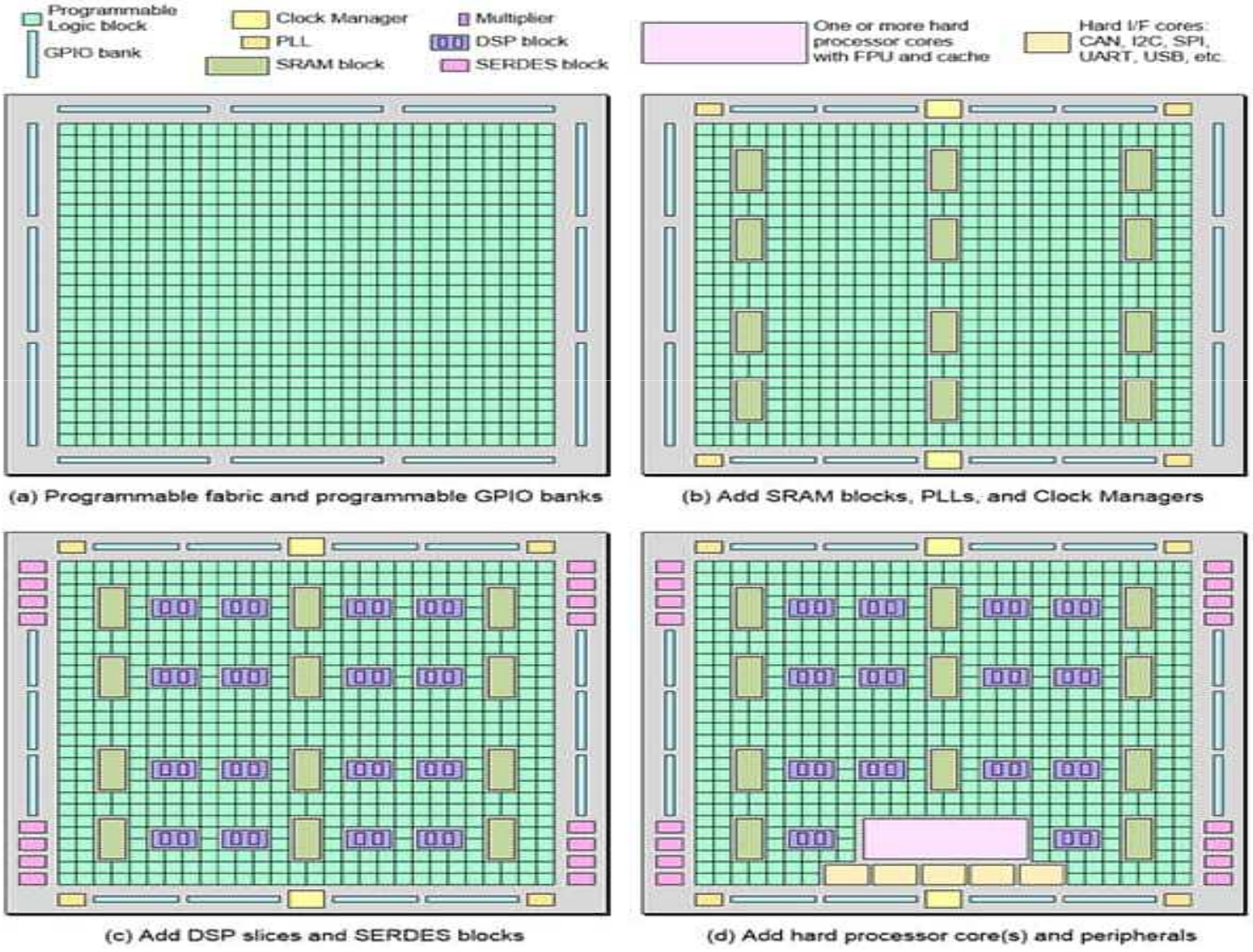


	cell5	cell4	Cell7	
	cell8	cell10	cell9	
		cell6		

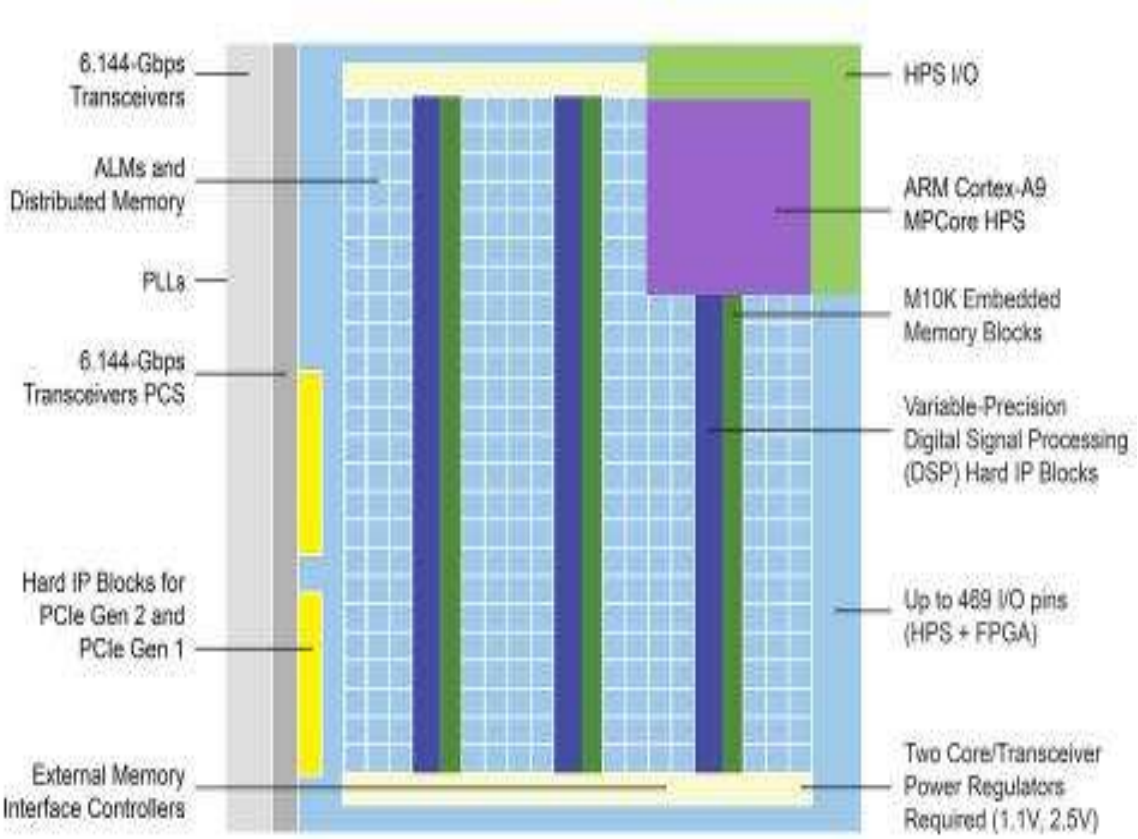


Evolution des FPGAs

Evolution des FPGAs



FPGA vs SoC



Exemple famille cyclone V:
Cores + FPGA + cellules dédiées
110K LEs (logic element)

Evolution des FPGAs



